Next Role

Find work that fits - complete source code

React + TypeScript + Tailwind CSS · Node + Express + MongoDB · Grok · Google OAuth · Electron

48 source files · generated 2026-10-10

Contents


```
.env.example
client/index.html
client/package.json
client/src/App.tsx
client/src/components/ExperienceCard.tsx
client/src/components/Footer.tsx
client/src/components/Header.tsx
client/src/components/Hero.tsx
client/src/components/Layout.tsx
client/src/components/LogoMark.tsx
client/src/components/RolesPanel.tsx
client/src/context/AuthContext.tsx
client/src/index.css
client/src/lib/api.ts
client/src/lib/content.ts
client/src/lib/slug.ts
client/src/main.tsx
client/src/pages/HomePage.tsx
client/src/pages/ProfilePage.tsx
client/src/pages/RoleDetailPage.tsx
client/src/pages/RolesPage.tsx
client/src/pages/SignInPage.tsx
client/src/types.ts
client/tsconfig.json
client/vite.config.ts
electron/main.cjs
nextrole.json
package.json
README.md
server/package.json
server/src/data.ts
server/src/db.ts
server/src/env.ts
server/src/index.ts
server/src/middleware/auth.ts
server/src/models.ts
server/src/routes/analyze.ts
server/src/routes/auth.ts
server/src/routes/profile.ts
server/src/routes/roles.ts
server/src/services/extract.ts
server/src/services/grok.ts
server/src/services/matcher.ts
server/src/services/ocr.ts
server/src/services/passwords.ts
server/src/store.ts
server/src/types.ts
server/tsconfig.json
```

.env.example

```
# ------------------------------------------------------------------
# Next Role environment variables
# Copy this file to `.env` in the project root, then fill in values.
# ------------------------------------------------------------------

# --- Server ---
PORT=4000
MONGODB_URI=mongodb://127.0.0.1:27017/nextrole
JWT_SECRET=change-me-to-a-long-random-string
CLIENT_ORIGIN=http://localhost:5173

# Google OAuth (Google Cloud Console -> OAuth client ID, Web application)
# Authorized JavaScript origin: http://localhost:5173
GOOGLE_CLIENT_ID=your-client-id.apps.googleusercontent.com

# xAI Grok API (https://console.x.ai) - optional, falls back to local matching when empty
GROK_API_KEY=
GROK_MODEL=grok-4-fast
GROK_API_URL=https://api.x.ai/v1/chat/completions

# --- Client (Vite only reads VITE_* variables) ---
VITE_GOOGLE_CLIENT_ID=your-client-id.apps.googleusercontent.com

```

client/index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Next Role - Find work that fits</title>
    <meta
      name="description"
      content="Bring your experience. We will surface roles that make sense for where you have been and where you coul…
    />
    <link
      rel="icon"
      href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 32 32'%3E%3Crect width='32' heig…
    />
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link
      href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&family=Instrument+Serif:ital@0;1&d…
      rel="stylesheet"
    />
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>

```

client/package.json

```
{
  "name": "nextrole-client",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc --noEmit && vite build",
    "preview": "vite preview",
    "typecheck": "tsc --noEmit"
  },
  "dependencies": {
    "@react-oauth/google": "^0.12.1",
    "lucide-react": "^0.446.0",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-router-dom": "^6.26.2"
  },
  "devDependencies": {
    "@tailwindcss/vite": "^4.1.11",
```

```
    "@types/node": "^20.16.10",
    "@types/react": "^18.3.10",
    "@types/react-dom": "^18.3.0",
    "@vitejs/plugin-react": "^4.3.2",
    "tailwindcss": "^4.1.11",
    "typescript": "^5.6.2",
    "vite": "^5.4.8"
  }
}

```

client/src/App.tsx

```
import { Navigate, Route, Routes } from 'react-router-dom';
import Layout from './components/Layout';
import HomePage from './pages/HomePage';
import ProfilePage from './pages/ProfilePage';
import RoleDetailPage from './pages/RoleDetailPage';
import RolesPage from './pages/RolesPage';
import SignInPage from './pages/SignInPage';

export default function App() {
  return (
    <Routes>
      <Route element={<Layout />}>
        <Route path="/" element={<HomePage />} />
        <Route path="/roles" element={<RolesPage />} />
        <Route path="/roles/:slug" element={<RoleDetailPage />} />
        <Route path="/profile" element={<ProfilePage />} />
        <Route path="/signin" element={<SignInPage />} />
        <Route path="/signup" element={<SignInPage initialMode="signup" />} />
        <Route path="*" element={<Navigate to="/" replace />} />
      </Route>
    </Routes>
  );
}

```

client/src/components/ExperienceCard.tsx

```
import { useLayoutEffect, useRef, useState } from 'react';
import { Check, FileText, Upload } from 'lucide-react';
import { content } from '../lib/content';
import type { AnalyzeStatus } from '../types';

type Tab = 'upload' | 'paste';

type ExperienceCardProps = {
  status: AnalyzeStatus;
  error: string | null;
  fileName: string | null;
  onFile: (file: File) => void;
  onText: (text: string) => void;
};

const MAX_BYTES = 10 * 1024 * 1024;
const ACCEPTED = '.pdf,.docx,.txt,.text';
const ALLOWED_EXTENSIONS = ['.pdf', '.docx', '.txt', '.text'];

export default function ExperienceCard({ status, error, fileName, onFile, onText }: ExperienceCardProps) {
  const copy = content.experience;
  const [tab, setTab] = useState<Tab>('upload');
  const [dragging, setDragging] = useState(false);
  const [text, setText] = useState('');
  const [wantPicker, setWantPicker] = useState(false);
  const [fileError, setFileError] = useState<string | null>(null);
  const inputRef = useRef<HTMLInputElement>(null);

  const loading = status === 'loading';

  // Clicking the "Upload a file" tab opens the file browser right away.
  useLayoutEffect(() => {
    if (!wantPicker) return;
    const input = inputRef.current;
    if (input) {
      input.click();
      setWantPicker(false);
    }
  }, [wantPicker, tab]);

  function handleUploadTab() {
    if (loading) return;
    setTab('upload');
```

```
    setWantPicker(true);
  }

  /** Validates the picked file and explains anything we cannot read. */
  function pickFile(file: File | undefined | null) {
    if (!file || loading) return;
    const extension = `.${(file.name.split('.').pop() ?? '').toLowerCase()}`;
    if (file.size === 0) {
      setFileError('That file is empty. Please choose your resume file again.');
      return;
    }
    if (file.size > MAX_BYTES) {
      setFileError(
        `"${file.name}" is larger than 10 MB. Try exporting a smaller PDF, or use the ${copy.tabPaste}
tab.`,
      );
      return;
    }
    if (!ALLOWED_EXTENSIONS.includes(extension)) {
      setFileError(
        `"${file.name}" is a ${extension === '.' ? 'file without an extension' : extension + ' file'}.
Please choose a…
      );
      return;
    }
    setFileError(null);
    onFile(file);
  }

  function submitText() {
    if (loading) return;
    onText(text);
  }

  const tabClass = (active: boolean) =>
    [
      'relative -mb-px pb-3 text-[14.5px] font-semibold transition-colors',
      active ? 'text-white' : 'text-white/45 hover:text-white/75',
    ].join(' ');

  return (
    <section className="rounded-2xl bg-forest p-6 text-white shadow-
[0_24px_50px_-32px_rgba(13,44,32,0.85)]">
      <div className="flex items-center gap-3">
        <span className="grid h-7 w-7 shrink-0 place-items-center rounded-full border border-white/45
text-[12px] font…
          {copy.step}
        </span>
        <div>
          <p className="text-[10.5px] font-semibold uppercase tracking-[0.16em] text-white/55">
            {copy.label}
          </p>
          <p className="text-[21px] font-bold leading-tight">{copy.title}</p>
        </div>
      </div>

      <p className="mt-4 text-[14.5px] leading-relaxed text-white/70">{copy.description}</p>

      <div className="mt-5 flex gap-7 border-b border-white/15" role="tablist">
        <button
          type="button"
          role="tab"
          aria-selected={tab === 'upload'}
          onClick={handleUploadTab}
          className={tabClass(tab === 'upload')}
        >
          {copy.tabUpload}
          {tab === 'upload' && (
            <span className="absolute inset-x-0 -bottom-px h-0.5 rounded-full bg-leaf" />
          )}
        </button>
        <button
          type="button"
          role="tab"
          aria-selected={tab === 'paste'}
          onClick={() => setTab('paste')}
          className={tabClass(tab === 'paste')}
        >
          {copy.tabPaste}
          {tab === 'paste' && (
            <span className="absolute inset-x-0 -bottom-px h-0.5 rounded-full bg-leaf" />
          )}
        </button>
      </div>

      {tab === 'upload' ? (
        <div>
          <label
            onDragOver={(event) => {
```

```
 129|               event.preventDefault();
 130|               if (!loading) setDragging(true);
 131|             }}
 132|             onDragLeave={() => setDragging(false)}
 133|             onDrop={(event) => {
 134|               event.preventDefault();
 135|               setDragging(false);
 136|               pickFile(event.dataTransfer.files?.[0]);
 137|             }}
 138|             className={[
 139|               'mt-4 flex cursor-pointer flex-col items-center justify-center rounded-xl bg-sand px-4 py-9
text-center …
 140|               dragging ? 'ring-2 ring-clay' : 'hover:bg-[#e9e6d8]',
 141|               loading ? 'pointer-events-none opacity-60' : '',
 142|             ].join(' ')}
 143|           >
 144|             <input
 145|               ref={inputRef}
 146|               type="file"
 147|               accept={ACCEPTED}
 148|               className="sr-only"
 149|               onChange={(event) => {
 150|                 pickFile(event.target.files?.[0]);
 151|                 event.target.value = '';
 152|               }}
 153|             />
 154|             <span className="grid h-10 w-10 place-items-center rounded-lg bg-white/70 text-forest shadow-
sm">
 155|               <Upload size={18} strokeWidth={2.2} />
 156|             </span>
 157|             <span className="mt-3 text-[13.5px] text-fern">
 158|               {dragging ? (
 159|                 <span className="font-semibold text-ink">{copy.dropHere}</span>
 160|               ) : (
 161|                 <>
 162|                   {copy.browseLead}{' '}
 163|                   <span className="font-bold text-ink underline underline-offset-2">
 164|                     {copy.browseLink}
 165|                   </span>
 166|                 </>
 167|               )}
 168|             </span>
 169|             <span className="mt-1.5 text-[11.5px] text-moss">{copy.fileTypes}</span>
 170|           </label>
 171|
 172|           {fileError && (
 173|             <div
 174|               role="alert"
 175|               className="mt-3 rounded-lg border border-[#ffb494]/50 bg-[#ffb494]/15 px-3 py-2.5 text-
[13px] leading-sn…
 176|             >
 177|               {fileError}
 178|             </div>
 179|           )}
 180|         </div>
 181|       ) : (
 182|         <div className="mt-4">
 183|           <textarea
 184|             value={text}
 185|             onChange={(event) => setText(event.target.value)}
 186|             rows={7}
 187|             placeholder={copy.pastePlaceholder}
 188|             className="w-full resize-none rounded-xl bg-sand p-4 text-[13.5px] leading-relaxed text-ink
placeholder:te…
 189|           />
 190|           <button
 191|             type="button"
 192|             onClick={submitText}
 193|             disabled={loading || text.trim().length === 0}
 194|             className="mt-3 w-full rounded-lg bg-clay px-4 py-2.5 text-[14.5px] font-bold text-white
transition hover:…
 195|           >
 196|             {copy.submit}
 197|           </button>
 198|         </div>
 199|       )}
 200|
 201|       {loading && (
 202|         <div className="mt-4 flex items-center gap-2.5 text-[13.5px] text-white/75">
 203|           <span className="h-4 w-4 animate-spin rounded-full border-2 border-white/25 border-t-leaf" />
 204|           {copy.loading}
 205|         </div>
 206|       )}
 207|
 208|       {!loading && fileName && status === 'ready' && (
 209|         <div className="mt-4 flex items-center gap-2 rounded-lg bg-white/10 px-3 py-2 text-[12.5px] text-
white/80">
 210|           <FileText size={14} className="shrink-0" />
 211|           <span className="truncate">{fileName}</span>
```

```
          <Check size={14} className="ml-auto shrink-0 text-leaf" />
        </div>
      )}

      {!loading && error && (
        <div className="mt-4 rounded-lg border border-clay/40 bg-clay/15 px-3 py-2.5 text-[13px] leading-snug text-[#f…
          {error}
        </div>
      )}
    </section>
  );
}
```

client/src/components/Footer.tsx

```
import { content } from '../lib/content';

export default function Footer() {
  return (
    <footer className="border-t border-black/10 bg-cream">
      <div className="mx-auto flex max-w-[1240px] flex-col items-center justify-between gap-2 px-6 py-6 text-[13px] te…
        <p>{content.footer.text}</p>
        <p>{content.footer.rights}</p>
      </div>
    </footer>
  );
}
```

client/src/components/Header.tsx

```
import { Link, useLocation } from 'react-router-dom';
import { content } from '../lib/content';
import { useAuth } from '../context/AuthContext';
import LogoMark from './LogoMark';

export default function Header() {
  const { user, signOut } = useAuth();
  const { pathname } = useLocation();

  // Each section is highlighted only on its own pages: Home ("/") is the
  // landing interface, Explore roles covers /roles and its detail pages.
  const isActive = (path: string) =>
    path === '/' ? pathname === '/' : pathname === path || pathname.startsWith(`${path}/`);

  return (
    <header className="sticky top-0 z-40 border-b border-black/10 bg-cream/90 backdrop-blur-sm">
      <div className="mx-auto flex h-[68px] max-w-[1240px] items-center justify-between gap-4 px-5 sm:px-6">
        <Link to="/" className="flex items-center gap-2.5" aria-label="nextrole home">
          <LogoMark />
          <span className="text-[19px] leading-none tracking-tight">
            <span className="font-extrabold text-ink">next</span>
            <span className="font-medium text-moss">role</span>
          </span>
        </Link>

        <nav className="hidden h-full items-stretch gap-8 md:flex" aria-label="Main">
          {content.nav.map((item) => (
            <Link
              key={item.path}
              to={item.path}
              className={[
                '-mb-px flex items-center border-b-2 pb-0.5 text-[15px] transition-colors',
                isActive(item.path)
                  ? 'border-clay font-semibold text-ink'
                  : 'border-transparent text-fern hover:text-ink',
              ].join(' ')}
            >
              {item.label}
            </Link>
          ))}
        </nav>

        <div className="flex items-center gap-3">
          {user ? (
            <>
              <Link
                to="/profile"
                className="hidden max-w-[180px] truncate text-[14px] font-medium text-fern hover:text-ink sm:block"
```

```
                title={user.email}
              >
                {user.name}
              </Link>
              <button
                type="button"
                onClick={signOut}
                className="rounded-lg border border-black/10 px-3.5 py-2 text-[14px] font-semibold text-
ink transition…
              >
                {content.actions.signOut}
              </button>
            </>
          ) : (
            <>
              <Link
                to="/signin"
                className="px-1 text-[15px] font-medium text-ink transition hover:opacity-70"
              >
                {content.actions.signIn}
              </Link>
              <Link
                to="/signup"
                className="rounded-lg bg-forest px-4 py-2.5 text-[14px] font-semibold text-white
transition hover:bg-f…
              >
                {content.actions.createAccount}
              </Link>
            </>
          )}
        </div>
      </div>

      {/* Mobile nav row */}
      <nav className="flex items-stretch gap-6 border-t border-black/5 px-5 sm:px-6 md:hidden" aria-
label="Mobile">
        {content.nav.map((item) => (
          <Link
            key={item.path}
            to={item.path}
            className={[
              'flex-1 border-b-2 py-2.5 text-center text-[14px] transition-colors',
              isActive(item.path)
                ? 'border-clay font-semibold text-ink'
                : 'border-transparent text-fern',
            ].join(' ')}
          >
            {item.label}
          </Link>
        ))}
      </nav>
    </header>
  );
}

```

client/src/components/Hero.tsx

```
import { content } from '../lib/content';

export default function Hero() {
  const { hero } = content;

  return (
    <section className="mx-auto max-w-[1240px] px-5 pb-8 pt-10 sm:px-6 sm:pt-14">
      <div className="flex items-start justify-between gap-8">
        <div className="max-w-[760px]">
          <div className="flex items-center gap-3">
            <span className="h-px w-7 bg-clay" aria-hidden="true" />
            <span className="text-[11.5px] font-semibold uppercase tracking-[0.16em] text-moss">
              {hero.eyebrow}
            </span>
          </div>

          <h1 className="mt-5 text-[clamp(34px,5.2vw,54px)] font-extrabold leading-[1.04] tracking-
[-0.03em] text-ink"…
            {hero.titleLead}
            <span className="font-serif text-[1.06em] font-normal italic text-clay">
              {hero.titleAccent}
            </span>
          </h1>

          <p className="mt-5 max-w-[560px] text-[16px] leading-relaxed text-fern">
            {hero.subtitle}
          </p>
        </div>
```

```
  28|
  29|         <div className="hidden shrink-0 items-start gap-3 rounded-xl border border-black/10 bg-white/70
px-4 py-3.5 sh…
  30|           <span className="grid h-7 w-7 shrink-0 place-items-center rounded-md bg-clay text-[13px] font-
bold text-whit…
  31|             {content.brand.mark}
  32|           </span>
  33|           <div>
  34|             <p className="text-[13.5px] font-bold leading-tight text-ink">{hero.privacyTitle}</p>
  35|             <p className="mt-0.5 text-[12.5px] leading-snug text-moss">{hero.privacyText}</p>
  36|           </div>
  37|         </div>
  38|       </div>
  39|     </section>
  40|   );
  41| }
  42|
```

client/src/components/Layout.tsx

```
   1| import { useEffect } from 'react';
   2| import { Outlet } from 'react-router-dom';
   3| import { content } from '../lib/content';
   4| import Header from './Header';
   5| import Footer from './Footer';
   6|
   7| export default function Layout() {
   8|   useEffect(() => {
   9|     document.title = content.title;
  10|   }, []);
  11|
  12|   return (
  13|     <div className="flex min-h-screen flex-col bg-cream">
  14|       <Header />
  15|       <main className="flex-1">
  16|         <Outlet />
  17|       </main>
  18|       <Footer />
  19|     </div>
  20|   );
  21| }
  22|
```

client/src/components/LogoMark.tsx

```
   1| type LogoMarkProps = {
   2|   size?: number;
   3| };
   4|
   5| /** The green "bars" mark shown next to the nextrole wordmark. */
   6| export default function LogoMark({ size = 30 }: LogoMarkProps) {
   7|   return (
   8|     <span
   9|       className="grid shrink-0 place-items-center rounded-[7px] bg-forest"
  10|       style={{ width: size, height: size }}
  11|       aria-hidden="true"
  12|     >
  13|       <svg width={size * 0.62} height={size * 0.62} viewBox="0 0 20 20" fill="none">
  14|         <rect x="1.5" y="11" width="4" height="7.5" rx="1.4" fill="#8fb877" />
  15|         <rect x="8" y="7" width="4" height="11.5" rx="1.4" fill="#b5d891" />
  16|         <rect x="14.5" y="2.5" width="4" height="16" rx="1.4" fill="#d9edb0" />
  17|       </svg>
  18|     </span>
  19|   );
  20| }
  21|
```

client/src/components/RolesPanel.tsx

```
   1| import { Plus } from 'lucide-react';
   2| import { Link } from 'react-router-dom';
   3| import { content } from '../lib/content';
   4| import { slugify } from '../lib/slug';
   5| import type { AnalyzeResult, AnalyzeStatus, RoleMatch } from '../types';
   6|
   7| type RolesPanelProps = {
   8|   status: AnalyzeStatus;
   9|   result: AnalyzeResult | null;
  10| };
  11|
```

```
function RoleCard({ role }: { role: RoleMatch }) {
  return (
    <Link
      to={`/roles/${slugify(role.title)}`}
      className="block rounded-xl border border-black/10 bg-white p-5 shadow-
[0_1px_2px_rgba(17,55,42,0.04)] transitio…
    >
      <div className="flex items-start justify-between gap-4">
        <div className="min-w-0">
          <h3 className="truncate text-[17px] font-bold text-ink">{role.title}</h3>
          <p className="mt-0.5 text-[12.5px] font-medium text-moss">{role.seniority}</p>
        </div>
        <span className="shrink-0 rounded-full bg-clay-100 px-2.5 py-1 text-[12px] font-bold text-clay-
deep">
          {role.matchPercent}
          {content.roles.matchSuffix}
        </span>
      </div>

      <p className="mt-3 text-[14px] leading-relaxed text-fern">{role.why}</p>

      {role.skills.length > 0 && (
        <div className="mt-3.5 flex flex-wrap gap-1.5">
          {role.skills.map((skill) => (
            <span
              key={skill}
              className="rounded-md bg-sand px-2 py-1 text-[11.5px] font-medium text-fern"
            >
              {skill}
            </span>
          ))}
        </div>
      )}
    </Link>
  );
}

export default function RolesPanel({ status, result }: RolesPanelProps) {
  const copy = content.roles;
  const loading = status === 'loading';
  const ready = status === 'ready' && result !== null;
  const count = ready ? result.roles.length : 0;

  return (
    <section aria-label={copy.title}>
      <div className="flex items-end justify-between gap-4">
        <div className="flex items-center gap-3">
          <span className="grid h-7 w-7 shrink-0 place-items-center rounded-full border border-clay text-
[12px] font-s…
            {copy.step}
          </span>
          <div>
            <p className="text-[10.5px] font-semibold uppercase tracking-[0.16em] text-moss">
              {copy.label}
            </p>
            <p className="text-[21px] font-bold leading-tight text-ink">{copy.title}</p>
          </div>
        </div>
        <p className="shrink-0 text-right leading-none">
          <span className="text-[30px] font-extrabold text-clay">{count}</span>{' '}
          <span className="text-[13px] text-moss">{copy.found}</span>
        </p>
      </div>

      {/* Idle notice bar */}
      {!loading && !ready && (
        <div className="mt-5 flex items-center gap-3 rounded-lg bg-bone px-4 py-3.5">
          <span className="grid h-6 w-6 shrink-0 place-items-center rounded-full bg-clay text-white">
            <Plus size={13} strokeWidth={3} />
          </span>
          <p className="text-[13.5px] leading-snug text-fern">{copy.notice}</p>
        </div>
      )}

      {/* Loading skeleton */}
      {loading && (
        <div className="mt-4 rounded-xl border border-black/10 bg-white/60 p-5">
          <p className="text-[14px] font-semibold text-ink">{copy.loadingTitle}</p>
          <div className="mt-4 space-y-3">
            {[0, 1, 2].map((row) => (
              <div key={row} className="animate-pulse space-y-2.5 rounded-lg bg-sand p-4">
                <div className="h-3.5 w-1/3 rounded bg-clay-100" />
                <div className="h-3 w-2/3 rounded bg-black/5" />
                <div className="h-3 w-1/2 rounded bg-black/5" />
              </div>
            ))}
          </div>
        </div>
      )}
```

```
  98|
  99|       {/* Empty state */}
 100|       {!loading && !ready && (
 101|         <div className="mt-4 grid min-h-[300px] place-items-center rounded-xl border-2 border-dashed
border-black/15 b…
 102|           <div>
 103|             <span className="mx-auto grid h-12 w-12 place-items-center rounded-xl bg-clay-100 text-[18px]
font-bold te…
 104|               {content.brand.mark}
 105|             </span>
 106|             <p className="mt-4 text-[17px] font-bold text-ink">{copy.emptyTitle}</p>
 107|             <p className="mt-1.5 text-[13.5px] text-moss">{copy.emptyText}</p>
 108|           </div>
 109|         </div>
 110|       )}
 111|
 112|       {/* Results */}
 113|       {ready && result && (
 114|         <div className="mt-5">
 115|           {result.summary && (
 116|             <div className="rounded-xl border border-black/10 bg-white p-5">
 117|               <p className="text-[11px] font-semibold uppercase tracking-[0.14em] text-moss">
 118|                 {copy.summaryLabel}
 119|               </p>
 120|               <p className="mt-1.5 text-[15px] leading-relaxed text-ink">{result.summary}</p>
 121|             </div>
 122|           )}
 123|
 124|           {result.skills.length > 0 && (
 125|             <div className="mt-4 rounded-xl border border-black/10 bg-white p-5">
 126|               <p className="text-[11px] font-semibold uppercase tracking-[0.14em] text-moss">
 127|                 {copy.skillsLabel}
 128|               </p>
 129|               <div className="mt-2.5 flex flex-wrap gap-1.5">
 130|                 {result.skills.map((skill) => (
 131|                   <span
 132|                     key={skill}
 133|                     className="rounded-md bg-sand px-2 py-1 text-[12px] font-medium text-fern"
 134|                   >
 135|                     {skill}
 136|                   </span>
 137|                 ))}
 138|               </div>
 139|             </div>
 140|           )}
 141|
 142|           <div className="mt-4 space-y-3.5">
 143|             {result.roles.map((role) => (
 144|               <RoleCard key={`${role.title}-${role.matchPercent}`} role={role} />
 145|             ))}
 146|           </div>
 147|
 148|           {result.source === 'local' && (
 149|             <p className="mt-4 text-[12.5px] text-moss">{copy.sourceLocal}</p>
 150|           )}
 151|         </div>
 152|       )}
 153|     </section>
 154|   );
 155| }
 156|
```

client/src/context/AuthContext.tsx

```
   1| import { createContext, useCallback, useContext, useEffect, useMemo, useState, type ReactNode } from
'react';
   2| import {
   3|   getStoredUser,
   4|   getToken,
   5|   googleSignIn,
   6|   localSignIn,
   7|   setSession,
   8|   validateSession,
   9| } from '../lib/api';
  10| import type { AuthUser } from '../types';
  11|
  12| type AuthContextValue = {
  13|   user: AuthUser | null;
  14|   token: string | null;
  15|   signInWithGoogle: (credential: string) => Promise<void>;
  16|   signInLocal: (input: {
  17|     mode: 'signin' | 'signup';
  18|     name?: string;
  19|     email: string;
  20|     password: string;
  21|   }) => Promise<void>;
```

```
  signOut: () => void;
};

const AuthContext = createContext<AuthContextValue | null>(null);

export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<AuthUser | null>(() => getStoredUser());
  const [token, setToken] = useState<string | null>(() => getToken());

  // A stored session can go stale (expired secret, cleared store): silently
  // drop it when the server rejects it so Sign in / Create account come back.
  useEffect(() => {
    if (!token) return;
    let cancelled = false;
    validateSession().then((ok) => {
      if (!ok && !cancelled) {
        setSession(null, null);
        setToken(null);
        setUser(null);
      }
    });
    return () => {
      cancelled = true;
    };
  }, [token]);

  const signInWithGoogle = useCallback(async (credential: string) => {
    const session = await googleSignIn(credential);
    setSession(session.token, session.user);
    setToken(session.token);
    setUser(session.user);
  }, []);

  const signInLocal = useCallback(
    async (input: { mode: 'signin' | 'signup'; name?: string; email: string; password: string }) => {
      const session = await localSignIn(input);
      setSession(session.token, session.user);
      setToken(session.token);
      setUser(session.user);
    },
    [],
  );

  const signOut = useCallback(() => {
    setSession(null, null);
    setToken(null);
    setUser(null);
  }, []);

  const value = useMemo(
    () => ({ user, token, signInWithGoogle, signInLocal, signOut }),
    [user, token, signInWithGoogle, signInLocal, signOut],
  );

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export function useAuth(): AuthContextValue {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth must be used inside <AuthProvider>');
  return ctx;
}

```

client/src/index.css

```
@import "tailwindcss";

@theme {
  --font-sans: "Inter", ui-sans-serif, system-ui, -apple-system, "Segoe UI", sans-serif;
  --font-serif: "Instrument Serif", Georgia, "Times New Roman", serif;

  /* Palette sampled from the Next Role design */
  --color-cream: #f7f6f1;
  --color-sand: #f1efe6;
  --color-bone: #edebe1;
  --color-forest: #123a2b;
  --color-forest-deep: #0d2c20;
  --color-ink: #11372a;
  --color-clay: #e2603c;
  --color-clay-deep: #c94e2c;
  --color-clay-100: #f9ded1;
  --color-clay-200: #f3c7b3;
  --color-sage: #9db4a7;
  --color-moss: #75897d;
  --color-fern: #64796e;
```

```
  --color-leaf: #cfe3a5;
}

@layer base {
  html {
    -webkit-font-smoothing: antialiased;
    -moz-osx-font-smoothing: grayscale;
  }

  body {
    @apply bg-cream font-sans text-ink;
  }

  ::selection {
    @apply bg-clay-100 text-ink;
  }
}

```

client/src/lib/api.ts

```
import type { AnalyzeResponse, AnalyzeResult, AuthUser, LibraryRole, SavedAnalysis } from '../types';

const API_BASE = (import.meta.env.VITE_API_URL as string | undefined) ?? '/api';

const TOKEN_KEY = 'nextrole.token';
const USER_KEY = 'nextrole.user';

export function getToken(): string | null {
  return localStorage.getItem(TOKEN_KEY);
}

export function getStoredUser(): AuthUser | null {
  const raw = localStorage.getItem(USER_KEY);
  if (!raw) return null;
  try {
    return JSON.parse(raw) as AuthUser;
  } catch {
    return null;
  }
}

export function setSession(token: string | null, user: AuthUser | null): void {
  if (token) localStorage.setItem(TOKEN_KEY, token);
  else localStorage.removeItem(TOKEN_KEY);
  if (user) localStorage.setItem(USER_KEY, JSON.stringify(user));
  else localStorage.removeItem(USER_KEY);
}

function authHeaders(): Record<string, string> {
  const token = getToken();
  return token ? { Authorization: `Bearer ${token}` } : {};
}

async function unwrap<T>(res: Response): Promise<T> {
  const data = (await res.json().catch(() => ({}))) as T & { error?: string };
  if (!res.ok) {
    throw new Error(data.error ?? `Request failed with status ${res.status}`);
  }
  return data;
}

/** Upload a resume file (PDF, DOCX, TXT) as raw bytes. */
export async function analyzeFile(file: File): Promise<AnalyzeResponse> {
  const res = await fetch(`${API_BASE}/analyze`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/octet-stream',
      'X-Filename': encodeURIComponent(file.name),
      ...authHeaders(),
    },
    body: file,
  });
  return unwrap<AnalyzeResponse>(res);
}

/** Analyse pasted resume text. */
export async function analyzeText(text: string): Promise<AnalyzeResponse> {
  const res = await fetch(`${API_BASE}/analyze`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      ...authHeaders(),
    },
    body: JSON.stringify({ text }),
  });
```

```
  return unwrap<AnalyzeResponse>(res);
}

export async function googleSignIn(credential: string): Promise<{ token: string; user: AuthUser }> {
  const res = await fetch(`${API_BASE}/auth/google`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ credential }),
  });
  return unwrap<{ token: string; user: AuthUser }>(res);
}

/** Password sign-in / sign-up with email (works with no external setup). */
export async function localSignIn(input: {
  mode: 'signin' | 'signup';
  name?: string;
  email: string;
  password: string;
}): Promise<{ token: string; user: AuthUser }> {
  const res = await fetch(`${API_BASE}/auth/local`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(input),
  });
  return unwrap<{ token: string; user: AuthUser }>(res);
}

const LAST_RESULT_KEY = 'nextrole.lastResult';

export type LastResult = AnalyzeResult & { fileName?: string };

/** Most recent analysis on this device (used for role detail matching). */
export function getLastResult(): LastResult | null {
  const raw = localStorage.getItem(LAST_RESULT_KEY);
  if (!raw) return null;
  try {
    return JSON.parse(raw) as LastResult;
  } catch {
    return null;
  }
}

export function setLastResult(result: LastResult): void {
  try {
    localStorage.setItem(LAST_RESULT_KEY, JSON.stringify(result));
  } catch {
    // Storage full or unavailable - results still render for this visit.
  }
}

export async function fetchLibraryRoles(): Promise<{ roles: LibraryRole[] }> {
  const res = await fetch(`${API_BASE}/roles`);
  return unwrap<{ roles: LibraryRole[] }>(res);
}

export async function fetchProfile(): Promise<{ analyses: SavedAnalysis[]; count: number }> {
  const res = await fetch(`${API_BASE}/profile`, { headers: authHeaders() });
  return unwrap<{ analyses: SavedAnalysis[]; count: number }>(res);
}

/** False only when the server explicitly rejects the stored token. */
export async function validateSession(): Promise<boolean> {
  try {
    const res = await fetch(`${API_BASE}/profile`, { headers: authHeaders() });
    if (res.ok) return true;
    return res.status !== 401 && res.status !== 403;
  } catch {
    return true; // Network problem: keep the session, do not sign out.
  }
}

export async function deleteAnalysis(id: string): Promise<{ ok: boolean }> {
  const res = await fetch(`${API_BASE}/profile/analysis/${encodeURIComponent(id)}`, {
    method: 'DELETE',
    headers: authHeaders(),
  });
  return unwrap<{ ok: boolean }>(res);
}

```

client/src/lib/content.ts

```
import rawContent from '../../../nextrole.json';

/**
 * Single source of truth for every piece of copy, title, skill and role
 * shown in the app: the `nextrole.json` file at the project root.
 */
export const content = rawContent;

export type Content = typeof content;

```

client/src/lib/slug.ts

```
/** URL-safe slug for a role title, e.g. "QA / Test Automation Engineer" -> "qa-test-automation-engineer". */
export function slugify(title: string): string {
  return title
    .toLowerCase()
    .replace(/&/g, ' and ')
    .replace(/[^a-z0-9]+/g, '-')
    .replace(/^-+|-+$/g, '');
}

```

client/src/main.tsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import { GoogleOAuthProvider } from '@react-oauth/google';
import App from './App';
import { AuthProvider } from './context/AuthContext';
import './index.css';

const googleClientId = (import.meta.env.VITE_GOOGLE_CLIENT_ID as string | undefined) ?? '';

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <GoogleOAuthProvider clientId={googleClientId}>
      <BrowserRouter>
        <AuthProvider>
          <App />
        </AuthProvider>
      </BrowserRouter>
    </GoogleOAuthProvider>
  </StrictMode>,
);

```

client/src/pages/HomePage.tsx

```
import { useCallback, useEffect, useState } from 'react';
import ExperienceCard from '../components/ExperienceCard';
import Hero from '../components/Hero';
import RolesPanel from '../components/RolesPanel';
import { useAuth } from '../context/AuthContext';
import { analyzeFile, analyzeText, fetchProfile, setLastResult } from '../lib/api';
import { content } from '../lib/content';
import type { AnalyzeResult, AnalyzeStatus } from '../types';

export default function HomePage() {
  const { token } = useAuth();
  const [status, setStatus] = useState<AnalyzeStatus>('idle');
  const [result, setResult] = useState<AnalyzeResult | null>(null);
  const [error, setError] = useState<string | null>(null);
  const [fileName, setFileName] = useState<string | null>(null);

  // When signed in, restore the most recent saved analysis.
  useEffect(() => {
    if (!token) return;
    let cancelled = false;
    fetchProfile()
      .then(({ analyses }) => {
        const latest = analyses[0];
        if (!cancelled && latest) {
          setResult({
```

```
            skills: latest.skills,
            summary: latest.summary,
            roles: latest.roles,
            source: latest.source,
          });
          setLastResult({
            skills: latest.skills,
            summary: latest.summary,
            roles: latest.roles,
            source: latest.source,
            fileName: latest.fileName,
          });
          setFileName(latest.fileName ?? null);
          setStatus('ready');
        }
      })
      .catch(() => undefined);
    return () => {
      cancelled = true;
    };
  }, [token]);

  const applyResponse = useCallback((response: Awaited<ReturnType<typeof analyzeFile>>) => {
    if (!response.ok) {
      throw new Error(response.error ?? content.experience.errors.server);
    }
    setResult({
      skills: response.skills,
      summary: response.summary,
      roles: response.roles,
      source: response.source,
    });
    setLastResult({
      skills: response.skills,
      summary: response.summary,
      roles: response.roles,
      source: response.source,
      fileName: response.fileName,
    });
    setStatus('ready');
    setError(null);
  }, []);

  const handleFile = useCallback(
    async (file: File) => {
      setStatus('loading');
      setError(null);
      setFileName(file.name);
      try {
        const response = await analyzeFile(file);
        applyResponse(response);
      } catch (cause) {
        setResult(null);
        setStatus('error');
        setError(cause instanceof Error ? cause.message : content.experience.errors.server);
      }
    },
    [applyResponse],
  );

  const handleText = useCallback(
    async (raw: string) => {
      const text = raw.trim();
      if (!text) {
        setError(content.experience.errors.empty);
        setStatus('error');
        return;
      }
      if (text.length < 60) {
        setError(content.experience.errors.tooShort);
        setStatus('error');
        return;
      }
      setStatus('loading');
      setError(null);
      setFileName(null);
      try {
        const response = await analyzeText(text);
        applyResponse(response);
      } catch (cause) {
        setResult(null);
        setStatus('error');
        setError(cause instanceof Error ? cause.message : content.experience.errors.server);
      }
    },
    [applyResponse],
  );

  return (
```

```
115|     <>
116|       <Hero />
117|
118|       <div className="mx-auto grid max-w-[1240px] items-start gap-7 px-5 pb-20 sm:px-6 lg:grid-cols-
[minmax(0,395px)_m…
119|         <ExperienceCard
120|           status={status}
121|           error={error}
122|           fileName={fileName}
123|           onFile={handleFile}
124|           onText={handleText}
125|         />
126|         <RolesPanel status={status} result={result} />
127|       </div>
128|     </>
129|   );
130| }
131|
```

client/src/pages/ProfilePage.tsx

```
  1| import { useCallback, useEffect, useState } from 'react';
  2| import { Link } from 'react-router-dom';
  3| import { useAuth } from '../context/AuthContext';
  4| import { deleteAnalysis, fetchProfile } from '../lib/api';
  5| import { content } from '../lib/content';
  6| import type { SavedAnalysis } from '../types';
  7|
  8| export default function ProfilePage() {
  9|   const copy = content.pages.profile;
 10|   const { user, token } = useAuth();
 11|   const [analyses, setAnalyses] = useState<SavedAnalysis[]>([]);
 12|   const [loading, setLoading] = useState(false);
 13|   const [error, setError] = useState<string | null>(null);
 14|
 15|   const load = useCallback(async () => {
 16|     if (!token) return;
 17|     setLoading(true);
 18|     setError(null);
 19|     try {
 20|       const data = await fetchProfile();
 21|       setAnalyses(data.analyses);
 22|     } catch (cause) {
 23|       setError(cause instanceof Error ? cause.message : 'Could not load your profile.');
 24|     } finally {
 25|       setLoading(false);
 26|     }
 27|   }, [token]);
 28|
 29|   useEffect(() => {
 30|     void load();
 31|   }, [load]);
 32|
 33|   async function remove(id: string) {
 34|     try {
 35|       await deleteAnalysis(id);
 36|       setAnalyses((current) => current.filter((item) => item.id !== id));
 37|     } catch (cause) {
 38|       setError(cause instanceof Error ? cause.message : 'Could not remove that analysis.');
 39|     }
 40|   }
 41|
 42|   if (!user) {
 43|     return (
 44|       <div className="mx-auto max-w-[720px] px-5 py-20 text-center sm:px-6">
 45|         <span className="mx-auto grid h-12 w-12 place-items-center rounded-xl bg-clay-100 text-[18px]
font-bold text-c…
 46|           {content.brand.mark}
 47|         </span>
 48|         <h1 className="mt-5 text-[clamp(28px,4vw,38px)] font-extrabold tracking-[-0.03em] text-ink">
 49|           {copy.title}
 50|         </h1>
 51|         <p className="mt-3 text-[15.5px] text-fern">{copy.signInPrompt}</p>
 52|         <Link
 53|           to="/signin"
 54|           className="mt-6 inline-block rounded-lg bg-forest px-5 py-3 text-[14.5px] font-semibold text-
white transitio…
 55|         >
 56|           {content.actions.signIn}
 57|         </Link>
 58|       </div>
 59|     );
 60|   }
 61|
 62|   const latest = analyses[0];
 63|
```

```
  return (
    <div className="mx-auto max-w-[1240px] px-5 py-12 sm:px-6">
      <div className="flex items-center gap-3">
        <span className="h-px w-7 bg-clay" aria-hidden="true" />
        <span className="text-[11.5px] font-semibold uppercase tracking-[0.16em] text-moss">
          {content.hero.eyebrow}
        </span>
      </div>

      <div className="mt-4 flex flex-col justify-between gap-4 sm:flex-row sm:items-end">
        <div>
          <h1 className="text-[clamp(30px,4vw,42px)] font-extrabold tracking-[-0.03em] text-ink">
            {copy.title}
          </h1>
          <p className="mt-2.5 text-[15.5px] text-fern">{copy.subtitle}</p>
        </div>
        <p className="text-[13.5px] text-moss">
          <span className="font-bold text-clay">{analyses.length}</span> {copy.analysesFound}
        </p>
      </div>

      <div className="mt-7 grid gap-5 lg:grid-cols-[minmax(0,340px)_minmax(0,1fr)]">
        {/* Account card */}
        <aside className="rounded-2xl bg-forest p-6 text-white">
          <div className="flex items-center gap-3">
            {user.picture ? (
              <img
                src={user.picture}
                alt=""
                className="h-11 w-11 rounded-full border border-white/30"
                referrerPolicy="no-referrer"
              />
            ) : (
              <span className="grid h-11 w-11 place-items-center rounded-full bg-clay text-[16px] font-
bold">
                {user.name.charAt(0).toUpperCase()}
              </span>
            )}
            <div className="min-w-0">
              <p className="truncate text-[16px] font-bold leading-tight">{user.name}</p>
              <p className="truncate text-[12.5px] text-white/60">{user.email}</p>
            </div>
          </div>

          {latest && (
            <div className="mt-5 rounded-xl bg-white/10 p-4">
              <p className="text-[10.5px] font-semibold uppercase tracking-[0.16em] text-white/55">
                {copy.topRole}
              </p>
              <p className="mt-1.5 text-[17px] font-bold leading-tight">
                {latest.roles[0]?.title ?? '?'}
              </p>
              {latest.roles[0] && (
                <p className="mt-0.5 text-[12.5px] text-leaf">
                  {latest.roles[0].matchPercent}
                  {content.roles.matchSuffix}
                </p>
              )}
            </div>
          )}

          <Link
            to="/"
            className="mt-5 block rounded-lg bg-clay px-4 py-2.5 text-center text-[14px] font-bold text-
white transiti…
          >
            {copy.openHome}
          </Link>
        </aside>

        {/* History */}
        <section>
          <h2 className="text-[17px] font-bold text-ink">{copy.historyTitle}</h2>

          {error && (
            <p className="mt-3 rounded-lg border border-clay/40 bg-clay-100 px-3 py-2 text-[13px] text-
clay-deep">
              {error}
            </p>
          )}

          {loading && (
            <div className="mt-4 space-y-3">
              {[0, 1].map((row) => (
                <div key={row} className="h-24 animate-pulse rounded-xl bg-bone" />
              ))}
            </div>
          )}
```

```
150|           {!loading && analyses.length === 0 && (
151|             <div className="mt-4 rounded-xl border-2 border-dashed border-black/15 bg-white/40 px-6 py-12
text-center"…
152|               <p className="text-[16px] font-bold text-ink">{copy.emptyTitle}</p>
153|               <p className="mt-1.5 text-[13.5px] text-moss">{copy.emptyText}</p>
154|               <Link
155|                 to="/"
156|                 className="mt-5 inline-block rounded-lg bg-forest px-4 py-2.5 text-[14px] font-semibold
text-white tra…
157|               >
158|                 {copy.openHome}
159|               </Link>
160|             </div>
161|           )}
162|
163|           {!loading && analyses.length > 0 && (
164|             <div className="mt-4 space-y-3.5">
165|               {analyses.map((item) => (
166|                 <article key={item.id} className="rounded-xl border border-black/10 bg-white p-5">
167|                   <div className="flex items-start justify-between gap-4">
168|                     <div className="min-w-0">
169|                       <p className="truncate text-[15.5px] font-bold text-ink">
170|                         {item.roles[0]?.title ?? content.roles.title}
171|                         {item.roles[0] && (
172|                           <span className="ml-2 rounded-full bg-clay-100 px-2 py-0.5 text-[11.5px] font-
bold text-clay…
173|                             {item.roles[0].matchPercent}
174|                             {content.roles.matchSuffix}
175|                           </span>
176|                         )}
177|                       </p>
178|                       <p className="mt-1 text-[12.5px] text-moss">
179|                         {new Date(item.createdAt).toLocaleString()} ? {item.roles.length}{' '}
180|                         {content.roles.found}
181|                         {item.fileName ? ` ? ${item.fileName}` : ''}
182|                       </p>
183|                     </div>
184|                     <button
185|                       type="button"
186|                       onClick={() => remove(item.id)}
187|                       className="shrink-0 rounded-lg border border-black/10 px-3 py-1.5 text-[12.5px]
font-semibold te…
188|                     >
189|                       {copy.deleteOne}
190|                     </button>
191|                   </div>
192|
193|                   {item.summary && (
194|                     <p className="mt-3 text-[13.5px] leading-relaxed text-fern">{item.summary}</p>
195|                   )}
196|
197|                   <div className="mt-3 flex flex-wrap gap-1.5">
198|                     {item.skills.slice(0, 10).map((skill) => (
199|                       <span
200|                         key={skill}
201|                         className="rounded-md bg-sand px-2 py-1 text-[11.5px] font-medium text-fern"
202|                       >
203|                         {skill}
204|                       </span>
205|                     ))}
206|                   </div>
207|                 </article>
208|               ))}
209|             </div>
210|           )}
211|         </section>
212|       </div>
213|     </div>
214|   );
215| }
216|
```

client/src/pages/RoleDetailPage.tsx

```
  1| import { useEffect, useMemo, useState } from 'react';
  2| import { Link, useParams } from 'react-router-dom';
  3| import { ArrowLeft, Check, Upload } from 'lucide-react';
  4| import { fetchLibraryRoles, getLastResult } from '../lib/api';
  5| import { content } from '../lib/content';
  6| import { slugify } from '../lib/slug';
  7| import type { LibraryRole } from '../types';
  8|
  9| /**
 10|  * One role, opened from any role card: shows every skill the role needs and -
 11|  * when a resume has been analysed on this device - which of them you already
 12|  * have and which are still to learn.
```

```
 */
export default function RoleDetailPage() {
  const copy = content.pages.roleDetail;
  const { slug } = useParams<{ slug: string }>();
  const [roles, setRoles] = useState<LibraryRole[]>(content.roleCatalog);
  const lastResult = useMemo(() => getLastResult(), []);

  useEffect(() => {
    let cancelled = false;
    fetchLibraryRoles()
      .then((data) => {
        if (!cancelled && data.roles.length > 0) setRoles(data.roles);
      })
      .catch(() => undefined);
    return () => {
      cancelled = true;
    };
  }, []);

  const role = roles.find((candidate) => slugify(candidate.title) === slug);
  const matchedRole = lastResult?.roles.find((candidate) => slugify(candidate.title) === slug);

  const ownedSkills = useMemo(() => {
    const owned = new Set<string>();
    for (const skill of lastResult?.skills ?? []) owned.add(skill.toLowerCase());
    return owned;
  }, [lastResult]);

  if (!role) {
    return (
      <div className="mx-auto max-w-[820px] px-5 py-16 sm:px-6">
        <div className="rounded-2xl border-2 border-dashed border-black/15 bg-white/60 px-6 py-14 text-
center">
          <p className="text-[18px] font-bold text-ink">{copy.notFound}</p>
          <Link
            to="/roles"
            className="mt-5 inline-flex items-center gap-2 rounded-xl bg-forest px-5 py-3 text-[14.5px]
font-bold text…
          >
            <ArrowLeft size={16} />
            {copy.notFoundCta}
          </Link>
        </div>
      </div>
    );
  }

  const has = role.skills.filter((skill) => ownedSkills.has(skill.toLowerCase()));
  const missing = role.skills.filter((skill) => !ownedSkills.has(skill.toLowerCase()));
  const coverage = role.skills.length > 0 ? Math.round((has.length / role.skills.length) * 100) : 0;

  return (
    <div className="mx-auto max-w-[900px] px-5 py-12 sm:px-6">
      <Link
        to="/roles"
        className="inline-flex items-center gap-2 text-[14px] font-semibold text-moss transition
hover:text-ink"
      >
        <ArrowLeft size={15} />
        {copy.backToRoles}
      </Link>

      {/* Role header */}
      <div className="mt-6 rounded-2xl border border-black/10 bg-white p-6 shadow-
[0_1px_2px_rgba(17,55,42,0.04)] sm:p…
        <div className="flex flex-wrap items-start justify-between gap-3">
          <div>
            <div className="flex items-center gap-3">
              <span className="h-px w-7 bg-clay" aria-hidden="true" />
              <span className="text-[11.5px] font-semibold uppercase tracking-[0.16em] text-moss">
                {content.hero.eyebrow}
              </span>
            </div>
            <h1 className="mt-3 text-[clamp(26px,3.6vw,38px)] font-extrabold tracking-[-0.03em] text-ink">
              {role.title}
            </h1>
            <p className="mt-1.5 text-[14px] font-semibold text-moss">{role.seniority}</p>
          </div>
          {matchedRole && (
            <span className="rounded-full bg-clay-100 px-3.5 py-1.5 text-[13.5px] font-bold text-clay-
deep">
              {matchedRole.matchPercent}
              {content.roles.matchSuffix}
            </span>
          )}
        </div>
        <p className="mt-4 text-[15.5px] leading-relaxed text-fern">{role.summary}</p>
        {matchedRole?.why && (
          <p className="mt-4 rounded-xl bg-bone px-4 py-3 text-[14px] leading-relaxed text-fern">
```

```
            {matchedRole.why}
          </p>
        )}
      </div>

      {/* Skills required */}
      <section className="mt-6 rounded-2xl border border-black/10 bg-white p-6 shadow-
[0_1px_2px_rgba(17,55,42,0.04)] …
        <p className="text-[11px] font-semibold uppercase tracking-[0.14em] text-moss">
          {copy.skillsTitle}
        </p>
        <p className="mt-1 text-[13.5px] text-moss">{copy.skillsNote}</p>

        <div className="mt-4 flex flex-wrap gap-2">
          {role.skills.map((skill) => {
            const owned = ownedSkills.has(skill.toLowerCase());
            return (
              <span
                key={skill}
                className={[
                  'inline-flex items-center gap-1.5 rounded-md px-2.5 py-1.5 text-[13px] font-medium',
                  owned
                    ? 'bg-[#e7f1d9] text-[#2f6b3f]'
                    : 'border border-black/10 bg-sand text-fern',
                ].join(' ')}
                title={lastResult ? (owned ? undefined : copy.skillsTitle) : undefined}
              >
                {owned && <Check size={13} strokeWidth={3} className="shrink-0" />}
                {skill}
              </span>
            );
          })}
        </div>

        {/* Fit summary against your resume */}
        <div className="mt-6 rounded-xl bg-bone p-5">
          {lastResult ? (
            <>
              <div className="flex items-baseline justify-between gap-3">
                <p className="text-[15px] font-bold text-ink">
                  {copy.youHave} <span className="text-clay">{has.length}</span> {copy.of}{' '}
                  {role.skills.length} {copy.skillsSuffix}
                </p>
                <p className="text-[14px] font-bold text-clay">{coverage}%</p>
              </div>
              <div className="mt-2.5 h-2 overflow-hidden rounded-full bg-black/10">
                <div
                  className="h-full rounded-full bg-forest transition-[width] duration-500"
                  style={{ width: `${coverage}%` }}
                  role="progressbar"
                  aria-valuenow={coverage}
                  aria-valuemin={0}
                  aria-valuemax={100}
                  aria-label={copy.fitTitle}
                />
              </div>
              {missing.length > 0 && (
                <p className="mt-3 text-[13.5px] leading-relaxed text-fern">
                  <span className="font-semibold text-ink">{missing.join(', ')}</span>
                </p>
              )}
            </>
          ) : (
            <p className="text-[14.5px] leading-relaxed text-fern">{copy.uploadPrompt}</p>
          )}
        </div>

        <Link
          to="/"
          className="mt-5 inline-flex items-center gap-2 rounded-xl bg-clay px-5 py-3 text-[14.5px] font-
bold text-whi…
        >
          <Upload size={16} />
          {copy.cta}
        </Link>
      </section>
    </div>
  );
}
```

client/src/pages/RolesPage.tsx

```tsx
import { useEffect, useMemo, useState } from 'react';
import { Link } from 'react-router-dom';
import { Check, ChevronDown, Upload } from 'lucide-react';
import { fetchLibraryRoles, getLastResult } from '../lib/api';
import { content } from '../lib/content';
import { slugify } from '../lib/slug';
import type { LibraryRole } from '../types';

export default function RolesPage() {
  const copy = content.pages.roles;
  const detailCopy = content.pages.roleDetail;
  const [roles, setRoles] = useState<LibraryRole[]>(content.roleCatalog);
  const [query, setQuery] = useState('');
  const [selectedTitle, setSelectedTitle] = useState('');
  const lastResult = useMemo(() => getLastResult(), []);

  useEffect(() => {
    let cancelled = false;
    fetchLibraryRoles()
      .then((data) => {
        if (!cancelled && data.roles.length > 0) setRoles(data.roles);
      })
      .catch(() => undefined);
    return () => {
      cancelled = true;
    };
  }, []);

  const selectedRole = roles.find((role) => role.title === selectedTitle) ?? null;

  // Match percentages from the latest resume analysis, keyed by role slug.
  const matchBySlug = useMemo(() => {
    const map = new Map<string, number>();
    for (const match of lastResult?.roles ?? []) {
      map.set(slugify(match.title), match.matchPercent);
    }
    return map;
  }, [lastResult]);

  const filtered = useMemo(() => {
    const q = query.trim().toLowerCase();
    const base = !q
      ? roles
      : roles.filter(
          (role) =>
            role.title.toLowerCase().includes(q) ||
            role.summary.toLowerCase().includes(q) ||
            role.skills.some((skill) => skill.toLowerCase().includes(q)),
        );
    // Once a resume has been analysed, the roles it matched float to the top.
    if (matchBySlug.size === 0) return base;
    return [...base].sort((a, b) => {
      const aMatch = matchBySlug.get(slugify(a.title)) ?? -1;
      const bMatch = matchBySlug.get(slugify(b.title)) ?? -1;
      return bMatch - aMatch;
    });
  }, [roles, query, matchBySlug]);

  const ownedSkills = useMemo(() => {
    const owned = new Set<string>();
    for (const skill of lastResult?.skills ?? []) owned.add(skill.toLowerCase());
    return owned;
  }, [lastResult]);

  const selectedHave = selectedRole
    ? selectedRole.skills.filter((skill) => ownedSkills.has(skill.toLowerCase()))
    : [];

  return (
    <div className="mx-auto max-w-[1240px] px-5 py-12 sm:px-6">
      <div className="flex flex-col justify-between gap-5 sm:flex-row sm:items-end">
        <div>
          <div className="flex items-center gap-3">
            <span className="h-px w-7 bg-clay" aria-hidden="true" />
            <span className="text-[11.5px] font-semibold uppercase tracking-[0.16em] text-moss">
              {content.hero.eyebrow}
            </span>
          </div>
          <h1 className="mt-4 text-[clamp(30px,4vw,42px)] font-extrabold tracking-[-0.03em] text-ink">
            {copy.title}
          </h1>
          <p className="mt-3 max-w-[560px] text-[15.5px] leading-relaxed text-fern">
            {copy.subtitle}
          </p>
          <p className="mt-4 text-[13px] text-moss">
```

```
              <span className="font-bold text-clay">{filtered.length}</span> {copy.countSuffix}
            </p>
            {matchBySlug.size > 0 && (
              <p className="mt-1.5 text-[12.5px] font-medium text-clay-deep">{copy.resumeSortHint}</p>
            )}
          </div>

          <input
            type="search"
            value={query}
            onChange={(event) => setQuery(event.target.value)}
            placeholder={copy.searchPlaceholder}
            className="w-full rounded-xl border border-black/10 bg-white px-4 py-3 text-[14.5px] text-ink
placeholder:te…
          />
        </div>

        <div className="mt-8 grid gap-4 sm:grid-cols-2 lg:grid-cols-3">
          {filtered.map((role) => (
            <Link
              key={role.title}
              to={`/roles/${slugify(role.title)}`}
              className="group flex flex-col rounded-xl border border-black/10 bg-white p-5 shadow-
[0_1px_2px_rgba(17,55…
            >
              <div className="flex items-start justify-between gap-3">
                <h2 className="text-[17px] font-bold leading-snug text-ink group-hover:text-clay-deep">
                  {role.title}
                </h2>
                <div className="flex shrink-0 flex-col items-end gap-1">
                  {matchBySlug.get(slugify(role.title)) !== undefined && (
                    <span className="rounded-md bg-clay px-2 py-1 text-[11px] font-bold text-white">
                      {matchBySlug.get(slugify(role.title))}% match
                    </span>
                  )}
                  <span className="rounded-md bg-sand px-2 py-1 text-[11px] font-semibold text-fern">
                    {role.seniority}
                  </span>
                </div>
              </div>
              <p className="mt-2.5 flex-1 text-[13.5px] leading-relaxed text-fern">{role.summary}</p>
              <div className="mt-4 flex flex-wrap gap-1.5">
                {role.skills.slice(0, 6).map((skill) => (
                  <span
                    key={skill}
                    className="rounded-md bg-clay-100 px-2 py-1 text-[11px] font-medium text-clay-deep"
                  >
                    {skill}
                  </span>
                ))}
              </div>
            </Link>
          ))}
        </div>

        {filtered.length === 0 && (
          <div className="mt-10 rounded-xl border-2 border-dashed border-black/15 bg-white/40 px-6 py-14
text-center">
            <p className="text-[16px] font-bold text-ink">No roles match ?{query}?.</p>
            <p className="mt-1.5 text-[13.5px] text-moss">Try a skill like React, SQL, or Figma.</p>
          </div>
        )}

        {/* ---- Role explorer: pick a role, see exactly what it takes ---- */}
        <section className="mt-10 rounded-2xl border border-black/10 bg-white p-6 shadow-
[0_1px_2px_rgba(17,55,42,0.04)]…
          <div className="flex items-center gap-3">
            <span className="h-px w-7 bg-clay" aria-hidden="true" />
            <span className="text-[11.5px] font-semibold uppercase tracking-[0.16em] text-moss">
              {copy.selectLabel}
            </span>
          </div>
          <p className="mt-2 text-[14px] text-fern">{copy.pickerHint}</p>

          <div className="relative mt-4 max-w-[440px]">
            <select
              aria-label={copy.selectLabel}
              value={selectedTitle}
              onChange={(event) => setSelectedTitle(event.target.value)}
              className="w-full appearance-none rounded-xl border border-black/10 bg-cream px-4 py-3 pr-11
text-[15px] f…
            >
              <option value="">{copy.selectPlaceholder}</option>
              {roles.map((role) => (
                <option key={role.title} value={role.title}>
                  {role.title}
                </option>
              ))}
            </select>
```

```
          <ChevronDown
            size={18}
            className="pointer-events-none absolute right-4 top-1/2 -translate-y-1/2 text-moss"
            aria-hidden="true"
          />
        </div>

        {selectedRole && (
          <div className="mt-5 rounded-xl bg-bone p-5">
            <div className="flex flex-wrap items-start justify-between gap-3">
              <div>
                <h2 className="text-[20px] font-extrabold tracking-[-0.02em] text-ink">
                  {selectedRole.title}
                </h2>
                <p className="mt-0.5 text-[13px] font-semibold text-moss">
                  {selectedRole.seniority}
                </p>
              </div>
              <Link
                to={`/roles/${slugify(selectedRole.title)}`}
                className="rounded-lg bg-forest px-4 py-2 text-[13.5px] font-bold text-white transition hover:bg-fores…
              >
                {copy.viewDetails}
              </Link>
            </div>

            <p className="mt-3 text-[14.5px] leading-relaxed text-fern">
              {selectedRole.summary}
            </p>

            <p className="mt-4 text-[11px] font-semibold uppercase tracking-[0.14em] text-moss">
              {detailCopy.skillsTitle}
            </p>
            <div className="mt-2.5 flex flex-wrap gap-2">
              {selectedRole.skills.map((skill) => {
                const owned = ownedSkills.has(skill.toLowerCase());
                return (
                  <span
                    key={skill}
                    className={[
                      'inline-flex items-center gap-1.5 rounded-md px-2.5 py-1.5 text-[13px] font-medium',
                      lastResult && owned
                        ? 'bg-[#e7f1d9] text-[#2f6b3f]'
                        : 'border border-black/10 bg-white text-fern',
                    ].join(' ')}
                  >
                    {lastResult && owned && <Check size={13} strokeWidth={3} className="shrink-0" />}
                    {skill}
                  </span>
                );
              })}
            </div>

            {lastResult ? (
              <p className="mt-4 text-[13.5px] text-fern">
                <span className="font-bold text-clay">
                  {detailCopy.youHave} {selectedHave.length} {detailCopy.of}{' '}
                  {selectedRole.skills.length}
                </span>{' '}
                {detailCopy.skillsSuffix}
                {selectedHave.length > 0 && (
                  <>
                    {' '}
                    - {selectedHave.join(', ')}
                  </>
                )}
              </p>
            ) : (
              <p className="mt-4 flex items-center gap-2 text-[13.5px] text-fern">
                <Upload size={14} className="shrink-0 text-clay" />
                {detailCopy.uploadPrompt}
              </p>
            )}
          </div>
        )}
      </section>
    </div>
  );
}

```

client/src/pages/SignInPage.tsx

```
  1| import { useState, type FormEvent } from 'react';
  2| import { Navigate, useNavigate } from 'react-router-dom';
  3| import { GoogleLogin } from '@react-oauth/google';
  4| import { useAuth } from '../context/AuthContext';
  5| import { content } from '../lib/content';
  6|
  7| const googleClientId = (import.meta.env.VITE_GOOGLE_CLIENT_ID as string | undefined) ?? '';
  8| const EMAIL_PATTERN = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  9|
 10| type Mode = 'signin' | 'signup';
 11|
 12| export default function SignInPage({ initialMode = 'signin' }: { initialMode?: Mode }) {
 13|   const copy = content.pages.signin;
 14|   const { user, signInWithGoogle, signInLocal } = useAuth();
 15|   const navigate = useNavigate();
 16|   const [mode, setMode] = useState<Mode>(initialMode);
 17|   const [error, setError] = useState<string | null>(null);
 18|   const [busy, setBusy] = useState(false);
 19|   const [name, setName] = useState('');
 20|   const [email, setEmail] = useState('');
 21|   const [password, setPassword] = useState('');
 22|
 23|   if (user) {
 24|     return <Navigate to="/profile" replace />;
 25|   }
 26|
 27|   const signingUp = mode === 'signup';
 28|
 29|   function switchMode(next: Mode) {
 30|     setMode(next);
 31|     setError(null);
 32|   }
 33|
 34|   function handleLocalSubmit(event: FormEvent) {
 35|     event.preventDefault();
 36|     if (busy) return;
 37|
 38|     if (!EMAIL_PATTERN.test(email.trim())) {
 39|       setError('Please enter a valid email address.');
 40|       return;
 41|     }
 42|     if (password.length < 6) {
 43|       setError('Your password must be at least 6 characters long.');
 44|       return;
 45|     }
 46|     if (signingUp && name.trim().length < 2) {
 47|       setError('Please enter your name.');
 48|       return;
 49|     }
 50|
 51|     setBusy(true);
 52|     setError(null);
 53|     signInLocal({
 54|       mode,
 55|       ...(signingUp ? { name: name.trim() } : {}),
 56|       email: email.trim().toLowerCase(),
 57|       password,
 58|     })
 59|       .then(() => navigate(signingUp ? '/profile' : '/profile'))
 60|       .catch((cause: unknown) =>
 61|         setError(cause instanceof Error ? cause.message : 'Sign in failed.'),
 62|       )
 63|       .finally(() => setBusy(false));
 64|   }
 65|
 66|   return (
 67|     <div className="mx-auto flex max-w-[560px] flex-col items-center px-5 py-16 text-center sm:px-6">
 68|       <span className="grid h-12 w-12 place-items-center rounded-xl bg-forest text-[18px] font-bold text-
leaf">
 69|         {content.brand.mark}
 70|       </span>
 71|
 72|       <div className="mt-5 flex items-center gap-3">
 73|         <span className="h-px w-6 bg-clay" aria-hidden="true" />
 74|         <span className="text-[11.5px] font-semibold uppercase tracking-[0.16em] text-moss">
 75|           {content.hero.eyebrow}
 76|         </span>
 77|         <span className="h-px w-6 bg-clay" aria-hidden="true" />
 78|       </div>
 79|
 80|       <h1 className="mt-4 text-[clamp(30px,4.5vw,40px)] font-extrabold tracking-[-0.03em] text-ink">
 81|         {signingUp ? copy.signupTitle : copy.title}
 82|       </h1>
 83|       <p className="mt-3 text-[15.5px] leading-relaxed text-fern">
 84|         {signingUp ? copy.signupSubtitle : copy.subtitle}
```

```
      </p>

      <div className="mt-8 w-full rounded-2xl border border-black/10 bg-white p-7 text-left shadow-
[0_1px_2px_rgba(17,…
        {/* Google OAuth - appears automatically once VITE_GOOGLE_CLIENT_ID is set */}
        {googleClientId &&
          (busy ? (
            <div className="flex justify-center py-2">
              <span className="h-6 w-6 animate-spin rounded-full border-2 border-black/10 border-t-clay" /
>
            </div>
          ) : (
            <div className="flex justify-center">
              <GoogleLogin
                theme="outline"
                size="large"
                text="continue_with"
                width={300}
                onSuccess={(credentialResponse) => {
                  const credential = credentialResponse.credential;
                  if (!credential) {
                    setError('Google did not return a credential. Please try again.');
                    return;
                  }
                  setBusy(true);
                  setError(null);
                  signInWithGoogle(credential)
                    .then(() => navigate('/profile'))
                    .catch((cause: unknown) =>
                      setError(cause instanceof Error ? cause.message : 'Sign in failed.'),
                    )
                    .finally(() => setBusy(false));
                }}
                onError={() => setError('Google sign-in failed. Please try again.')}
              />
            </div>
          ))}

        {googleClientId && (
          <div className="mt-5 flex items-center gap-3" aria-hidden="true">
            <span className="h-px flex-1 bg-black/10" />
            <span className="text-[12px] font-semibold uppercase tracking-[0.14em] text-moss">
              {copy.or}
            </span>
            <span className="h-px flex-1 bg-black/10" />
          </div>
        )}

        {/* Password sign-in / sign-up */}
        <form onSubmit={handleLocalSubmit} className="mt-4 space-y-3">
          {signingUp && (
            <div>
              <label className="mb-1.5 block text-[13px] font-semibold text-fern" htmlFor="auth-name">
                {copy.namePlaceholder}
              </label>
              <input
                id="auth-name"
                type="text"
                autoComplete="name"
                value={name}
                onChange={(event) => setName(event.target.value)}
                placeholder={copy.namePlaceholder}
                className="w-full rounded-xl border border-black/10 bg-cream px-4 py-3 text-[14.5px] text-
ink placehol…
              />
            </div>
          )}
          <div>
            <label className="mb-1.5 block text-[13px] font-semibold text-fern" htmlFor="auth-email">
              {copy.emailPlaceholder}
            </label>
            <input
              id="auth-email"
              type="email"
              autoComplete="email"
              value={email}
              onChange={(event) => setEmail(event.target.value)}
              placeholder={copy.emailPlaceholder}
              className="w-full rounded-xl border border-black/10 bg-cream px-4 py-3 text-[14.5px] text-
ink placeholde…
            />
          </div>
          <div>
            <label
              className="mb-1.5 block text-[13px] font-semibold text-fern"
              htmlFor="auth-password"
            >
              {copy.passwordPlaceholder}
            </label>
```

```
170|             <input
171|               id="auth-password"
172|               type="password"
173|               autoComplete={signingUp ? 'new-password' : 'current-password'}
174|               value={password}
175|               onChange={(event) => setPassword(event.target.value)}
176|               placeholder={copy.passwordPlaceholder}
177|               className="w-full rounded-xl border border-black/10 bg-cream px-4 py-3 text-[14.5px] text-
ink placeholde…
178|             />
179|             <p className="mt-1.5 text-[12px] text-moss">{copy.passwordHint}</p>
180|           </div>
181|           <button
182|             type="submit"
183|             disabled={busy}
184|             className="w-full rounded-xl bg-forest px-4 py-3 text-[15px] font-bold text-white transition
hover:bg-fore…
185|           >
186|             {busy ? '...' : signingUp ? copy.signupCta : copy.signinCta}
187|           </button>
188|           <p className="text-[12.5px] leading-snug text-moss">{copy.localNote}</p>
189|         </form>
190|
191|         {error && (
192|           <p
193|             role="alert"
194|             className="mt-4 rounded-lg border border-clay/40 bg-clay-100 px-3 py-2 text-[13px] text-clay-
deep"
195|           >
196|             {error}
197|           </p>
198|         )}
199|
200|         {/* Switch between sign in and sign up */}
201|         <p className="mt-5 text-[13.5px] text-fern">
202|           {signingUp ? copy.toSignin : copy.toSignup}{' '}
203|           <button
204|             type="button"
205|             onClick={() => switchMode(signingUp ? 'signin' : 'signup')}
206|             className="font-bold text-clay underline underline-offset-2 hover:text-clay-deep"
207|           >
208|             {signingUp ? copy.signinCta : copy.signupCta}
209|           </button>
210|         </p>
211|
212|         <p className="mt-6 text-[13px] leading-relaxed text-moss">{copy.note}</p>
213|         <p className="mt-2 text-[13px] text-moss">{copy.noAccount}</p>
214|       </div>
215|     </div>
216|   );
217| }
218|
```

client/src/types.ts

```
  1| export type RoleMatch = {
  2|   title: string;
  3|   matchPercent: number;
  4|   seniority: string;
  5|   summary: string;
  6|   why: string;
  7|   skills: string[];
  8| };
  9|
 10| export type AnalyzeResult = {
 11|   skills: string[];
 12|   summary: string;
 13|   roles: RoleMatch[];
 14|   source: 'grok' | 'local';
 15| };
 16|
 17| export type AnalyzeResponse = AnalyzeResult & {
 18|   ok: boolean;
 19|   saved: boolean;
 20|   fileName?: string;
 21|   error?: string;
 22| };
 23|
 24| export type AnalyzeStatus = 'idle' | 'loading' | 'ready' | 'error';
 25|
 26| export type AuthUser = {
 27|   id: string;
 28|   name: string;
 29|   email: string;
 30|   picture?: string;
 31| };
```

```

export type SavedAnalysis = {
  id: string;
  fileName?: string;
  summary: string;
  skills: string[];
  roles: RoleMatch[];
  source: 'grok' | 'local';
  createdAt: string;
};

export type LibraryRole = {
  title: string;
  seniority: string;
  summary: string;
  skills: string[];
};

```

client/tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "moduleResolution": "bundler",
    "skipLibCheck": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "react-jsx",
    "strict": true,
    "noFallthroughCasesInSwitch": true,
    "allowImportingTsExtensions": true,
    "types": ["vite/client", "node"]
  },
  "include": ["src", "vite.config.ts", "../nextrole.json"]
}

```

client/vite.config.ts

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import tailwindcss from '@tailwindcss/vite';
import { fileURLToPath } from 'node:url';

// Project root (one level above /client) so `nextrole.json` can be imported directly.
const projectRoot = fileURLToPath(new URL('..', import.meta.url));

export default defineConfig({
  plugins: [react(), tailwindcss()],
  // Load the shared .env from the project root.
  envDir: projectRoot,
  server: {
    port: 5173,
    proxy: {
      '/api': 'http://localhost:4000',
    },
    fs: {
      allow: [projectRoot],
    },
  },
  build: {
    outDir: 'dist',
    emptyOutDir: true,
  },
});

```

electron/main.cjs

```
/**
 * Next Role desktop app (Electron).
 *
 * Starts the bundled Node API server in-process (it also serves the built
 * client from client/dist), waits for its health endpoint, then opens the
 * app in a clean browser window. One double-clickable app, no terminals.
```

```
 */
const { app, BrowserWindow, shell } = require('electron');
const path = require('node:path');
const http = require('node:http');

const PORT = Number(process.env.NEXTROLE_PORT || 4173);
const APP_URL = `http://127.0.0.1:${PORT}`;

function startServer() {
  try {
    // Must happen before the server reads env: it binds process.env.PORT.
    process.env.PORT = String(PORT);
    require(path.join(__dirname, '..', 'server', 'dist', 'index.js'));
  } catch (error) {
    console.error('[desktop] failed to start the bundled server:', error);
  }
}

function waitForServer(retriesLeft = 120) {
  return new Promise((resolve) => {
    const attempt = (left) => {
      const req = http.get(`${APP_URL}/api/health`, (res) => {
        res.resume();
        if (res.statusCode === 200) {
          resolve(true);
          return;
        }
        retry(left);
      });
      req.on('error', () => retry(left));
      req.setTimeout(1500, () => {
        req.destroy();
        retry(left);
      });
    };
    const retry = (left) => {
      if (left <= 0) {
        resolve(false);
        return;
      }
      setTimeout(() => attempt(left - 1), 250);
    };
    attempt(retriesLeft);
  });
}

function createWindow() {
  const win = new BrowserWindow({
    width: 1320,
    height: 900,
    minWidth: 980,
    minHeight: 640,
    backgroundColor: '#f7f6f1',
    autoHideMenuBar: true,
    title: 'Next Role',
    webPreferences: {
      contextIsolation: true,
      nodeIntegration: false,
      sandbox: true,
    },
  });
  win.setMenuBarVisibility(false);
  win.loadURL(APP_URL);
  // Open external links (Google, docs) in the default browser, not in-app.
  win.webContents.setWindowOpenHandler(({ url }) => {
    if (url.startsWith('http')) shell.openExternal(url);
    return { action: 'deny' };
  });
  return win;
}

app.whenReady().then(async () => {
  startServer();
  const ready = await waitForServer();
  if (!ready) {
    console.error('[desktop] server never became healthy; loading anyway');
  }
  createWindow();
});

app.on('window-all-closed', () => {
  app.quit();
});

```

nextrole.json

```
{
  "title": "Next Role - Find work that fits",
  "brand": {
    "name": "nextrole",
    "mark": "N",
    "tagline": "Find work that fits you."
  },
  "nav": [
    {
      "label": "Home",
      "path": "/"
    },
    {
      "label": "Explore roles",
      "path": "/roles"
    },
    {
      "label": "Your profile",
      "path": "/profile"
    }
  ],
  "actions": {
    "signIn": "Sign in",
    "createAccount": "Create account",
    "signOut": "Sign out"
  },
  "hero": {
    "eyebrow": "YOUR NEXT CHAPTER, MADE CLEARER",
    "titleLead": "Find work that ",
    "titleAccent": "fits you.",
    "subtitle": "Bring your experience. We will surface roles that make sense for where you have been and where you co…
    "privacyTitle": "Your profile stays yours.",
    "privacyText": "Matching happens on this device."
  },
  "experience": {
    "step": "1",
    "label": "START WITH YOU",
    "title": "Your experience",
    "description": "Use your LinkedIn profile PDF, or paste the text from your profile.",
    "tabUpload": "Upload a file",
    "tabPaste": "Paste text",
    "browseLead": "or",
    "browseLink": "browse files",
    "fileTypes": "PDF, Word document, or plain text, up to 10 MB",
    "pastePlaceholder": "Paste your resume text, LinkedIn summary, or work history here...",
    "submit": "Show my roles",
    "dropHere": "Release to upload your file",
    "loading": "Reading your experience...",
    "ready": "Ready to analyse",
    "errors": {
      "empty": "Add your resume text or a file first.",
      "tooShort": "That looks too short. Paste at least a few lines about your experience.",
      "server": "Something went wrong while reading your resume. Please try again."
    }
  },
  "roles": {
    "step": "2",
    "label": "POSSIBLE NEXT MOVES",
    "title": "Roles for you",
    "found": "roles found",
    "notice": "Add your profile to see roles matched to your experience.",
    "emptyTitle": "Your matches will show up here",
    "emptyText": "Upload a profile or paste your experience to get started.",
    "loadingTitle": "Matching roles to your skills...",
    "skillsLabel": "Skills we found in your resume",
    "summaryLabel": "Your profile, in one line",
    "matchSuffix": "% match",
    "sourceLocal": "Local match - add a Grok API key for deeper AI analysis.",
    "retry": "Try again"
  },
  "pages": {
    "roles": {
      "title": "Explore roles",
      "subtitle": "Every role in the Next Role library, and the skills each one leans on.",
      "searchPlaceholder": "Search by role or skill...",
      "countSuffix": "roles in the library",
      "selectLabel": "Select a role to explore",
      "selectPlaceholder": "Choose a role to see what it takes...",
      "viewDetails": "Open full role details",
      "pickerHint": "Pick any role below to see the skills you need to apply for it.",
      "resumeSortHint": "Roles that match your resume are shown first - strongest fit at the top."
    },
    "profile": {
      "title": "Your profile",
```

```
      "subtitle": "Your saved matches, in one place.",
      "historyTitle": "Previous analyses",
      "emptyTitle": "No saved matches yet.",
      "emptyText": "Upload your resume on the home page and your matches will appear here.",
      "signInPrompt": "Sign in to keep your matches across devices.",
      "analysesFound": "saved analyses",
      "topRole": "Top match",
      "deleteOne": "Remove",
      "openHome": "Analyse a resume"
    },
    "signin": {
      "title": "Sign in",
      "subtitle": "Keep your matches, skills, and history together across devices.",
      "note": "We only use your name and email. Your resume text is never shared with anyone else.",
      "noAccount": "New here? Enter your details below to create your account - it takes seconds.",
      "or": "or",
      "namePlaceholder": "Your name",
      "emailPlaceholder": "Email address",
      "passwordPlaceholder": "Password",
      "passwordHint": "Use at least 6 characters.",
      "continue": "Continue",
      "signinCta": "Sign in",
      "signupCta": "Sign up",
      "signupTitle": "Sign up",
      "signupSubtitle": "Create your account with a name, email and password - it takes seconds.",
      "toSignup": "New here? Create an account",
      "toSignin": "Already have an account? Sign in",
      "localNote": "Your password is stored securely (hashed) and is never shared."
    },
    "roleDetail": {
      "backToRoles": "All roles",
      "skillsTitle": "Skills you need to apply",
      "skillsNote": "These are the skills this role leans on most.",
      "fitTitle": "Your fit for this role",
      "youHave": "You have",
      "of": "of",
      "skillsSuffix": "skills",
      "uploadPrompt": "Upload your resume and we will show which of these skills you already have.",
      "matchedSuffix": "% match with your resume",
      "cta": "Check my resume for this role",
      "notFound": "That role is not in the library yet.",
      "notFoundCta": "Back to all roles"
    }
  },
  "footer": {
    "text": "Your next chapter, made clearer.",
    "rights": "? 2026 nextrole"
  },
  "skillCatalog": [
    "JavaScript",
    "TypeScript",
    "Python",
    "Java",
    "C++",
    "C#",
    "Go",
    "Rust",
    "R",
    "PHP",
    "Ruby",
    "Kotlin",
    "Swift",
    "HTML",
    "CSS",
    "SQL",
    "GraphQL",
    "REST APIs",
    "React",
    "Next.js",
    "Vue.js",
    "Angular",
    "Node.js",
    "Express",
    "Django",
    "Flask",
    "Spring Boot",
    "React Native",
    "Flutter",
    "Tailwind CSS",
    "Bootstrap",
    "jQuery",
    "MongoDB",
    "PostgreSQL",
    "MySQL",
    "Redis",
    "Elasticsearch",
    "Firebase",
    "Oracle",
    "Excel",
```

```
    "Power BI",
    "Tableau",
    "Google Sheets",
    "pandas",
    "NumPy",
    "Statistics",
    "Data Visualization",
    "ETL",
    "Data Modeling",
    "Data Warehousing",
    "Big Data",
    "Spark",
    "Hadoop",
    "Airflow",
    "Kafka",
    "Machine Learning",
    "Deep Learning",
    "TensorFlow",
    "PyTorch",
    "scikit-learn",
    "NLP",
    "Computer Vision",
    "Prompt Engineering",
    "AWS",
    "Azure",
    "Google Cloud",
    "Docker",
    "Kubernetes",
    "Terraform",
    "CI/CD",
    "Jenkins",
    "GitHub Actions",
    "Linux",
    "Nginx",
    "Monitoring",
    "Ansible",
    "Microservices",
    "Serverless",
    "Git",
    "GitHub",
    "Jira",
    "Figma",
    "Sketch",
    "Canva",
    "Photoshop",
    "Illustrator",
    "Premiere Pro",
    "After Effects",
    "Notion",
    "Slack",
    "Salesforce",
    "HubSpot",
    "SAP",
    "Microsoft Office",
    "PowerPoint",
    "Word",
    "Agile",
    "Scrum",
    "Kanban",
    "Roadmaps",
    "Product Strategy",
    "User Research",
    "Wireframing",
    "Prototyping",
    "Design Systems",
    "Accessibility",
    "UI/UX Design",
    "A/B Testing",
    "SEO",
    "Content Marketing",
    "Email Marketing",
    "Social Media",
    "Google Analytics",
    "Copywriting",
    "Technical Writing",
    "Documentation",
    "Communication",
    "Leadership",
    "Mentoring",
    "Stakeholder Management",
    "Project Management",
    "Negotiation",
    "Customer Success",
    "Onboarding",
    "Recruitment",
    "Payroll",
    "Financial Modeling",
    "Forecasting",
    "Accounting",
```

```
    "Risk Management",
    "Quality Assurance",
    "Testing",
    "Jest",
    "Selenium",
    "Cypress",
    "Playwright",
    "Unit Testing",
    "Performance Tuning",
    "Security",
    "Networking",
    "System Design",
    "Problem Solving",
    "Analytical Thinking",
    "Attention to Detail",
    "Time Management",
    "Teamwork",
    "Presentation Skills",
    "Budget Management",
    "Vendor Management",
    "Supply Chain",
    "Inventory Management",
    "Customer Support",
    "Technical Support",
    "CRM",
    "Business Analysis",
    "Requirements Gathering",
    "Process Improvement",
    "Change Management",
    "Business Intelligence",
    "Budgeting",
    "Payroll Processing",
    "Employment Law",
    "Interviewing"
  ],
  "skillRoles": {
    "React": [
      "Frontend Engineer",
      "Full-Stack Developer",
      "UI/UX Designer"
    ],
    "TypeScript": [
      "Frontend Engineer",
      "Backend Engineer",
      "Full-Stack Developer"
    ],
    "JavaScript": [
      "Frontend Engineer",
      "Full-Stack Developer",
      "Backend Engineer"
    ],
    "HTML": [
      "Frontend Engineer",
      "UI/UX Designer",
      "Technical Writer"
    ],
    "CSS": [
      "Frontend Engineer",
      "UI/UX Designer"
    ],
    "Tailwind CSS": [
      "Frontend Engineer",
      "UI/UX Designer"
    ],
    "Node.js": [
      "Backend Engineer",
      "Full-Stack Developer"
    ],
    "Express": [
      "Backend Engineer",
      "Full-Stack Developer"
    ],
    "MongoDB": [
      "Backend Engineer",
      "Full-Stack Developer",
      "Database Administrator"
    ],
    "SQL": [
      "Data Analyst",
      "Backend Engineer",
      "Database Administrator",
      "Business Analyst"
    ],
    "PostgreSQL": [
      "Backend Engineer",
      "Database Administrator"
    ],
    "MySQL": [
      "Backend Engineer",
```

```
      "Database Administrator"
    ],
    "Python": [
      "Data Analyst",
      "Data Scientist",
      "Backend Engineer",
      "Machine Learning Engineer"
    ],
    "Docker": [
      "DevOps Engineer",
      "Backend Engineer",
      "Cloud Engineer"
    ],
    "Kubernetes": [
      "DevOps Engineer",
      "Cloud Engineer"
    ],
    "AWS": [
      "Cloud Engineer",
      "DevOps Engineer",
      "Machine Learning Engineer"
    ],
    "Azure": [
      "Cloud Engineer",
      "DevOps Engineer"
    ],
    "Git": [
      "Frontend Engineer",
      "Backend Engineer",
      "Full-Stack Developer"
    ],
    "Figma": [
      "UI/UX Designer",
      "Product Manager"
    ],
    "UI/UX Design": [
      "UI/UX Designer"
    ],
    "Photoshop": [
      "Graphic Designer",
      "UI/UX Designer"
    ],
    "Illustrator": [
      "Graphic Designer"
    ],
    "Premiere Pro": [
      "Video Editor"
    ],
    "Machine Learning": [
      "Data Scientist",
      "Machine Learning Engineer"
    ],
    "TensorFlow": [
      "Machine Learning Engineer",
      "Data Scientist"
    ],
    "PyTorch": [
      "Machine Learning Engineer",
      "Data Scientist"
    ],
    "Statistics": [
      "Data Analyst",
      "Data Scientist"
    ],
    "Excel": [
      "Financial Analyst",
      "Data Analyst",
      "Operations Manager",
      "HR Generalist"
    ],
    "Power BI": [
      "Data Analyst",
      "Business Analyst"
    ],
    "Tableau": [
      "Data Analyst",
      "Business Analyst"
    ],
    "Agile": [
      "Project Manager",
      "Product Manager",
      "Engineering Manager"
    ],
    "Scrum": [
      "Project Manager",
      "Product Manager"
    ],
    "Roadmaps": [
      "Product Manager",
```

```
      "Engineering Manager"
    ],
    "SEO": [
      "Digital Marketing Specialist",
      "Content Strategist",
      "Growth Marketer"
    ],
    "Content Marketing": [
      "Content Strategist",
      "Digital Marketing Specialist"
    ],
    "Copywriting": [
      "Content Strategist",
      "Digital Marketing Specialist"
    ],
    "Google Analytics": [
      "Digital Marketing Specialist",
      "Growth Marketer"
    ],
    "CRM": [
      "Sales Development Representative",
      "Customer Success Manager",
      "Recruiter"
    ],
    "Salesforce": [
      "Sales Development Representative",
      "Customer Success Manager"
    ],
    "Communication": [
      "Project Manager",
      "Customer Success Manager",
      "Recruiter",
      "Technical Writer"
    ],
    "Leadership": [
      "Engineering Manager",
      "Operations Manager",
      "Project Manager"
    ],
    "Recruitment": [
      "Recruiter",
      "HR Generalist"
    ],
    "Payroll": [
      "HR Generalist"
    ],
    "Financial Modeling": [
      "Financial Analyst"
    ],
    "Accounting": [
      "Financial Analyst"
    ],
    "Testing": [
      "QA / Test Automation Engineer",
      "Full-Stack Developer"
    ],
    "Selenium": [
      "QA / Test Automation Engineer"
    ],
    "Jest": [
      "QA / Test Automation Engineer",
      "Frontend Engineer"
    ],
    "Security": [
      "Cybersecurity Analyst"
    ],
    "Networking": [
      "Cybersecurity Analyst",
      "Cloud Engineer"
    ],
    "Linux": [
      "DevOps Engineer",
      "Cybersecurity Analyst",
      "Cloud Engineer"
    ],
    "React Native": [
      "Mobile App Developer"
    ],
    "Flutter": [
      "Mobile App Developer"
    ],
    "Swift": [
      "Mobile App Developer"
    ],
    "Kotlin": [
      "Mobile App Developer"
    ],
    "Documentation": [
      "Technical Writer"
```

```
    ],
    "Technical Writing": [
      "Technical Writer"
    ],
    "Customer Support": [
      "Customer Success Manager",
      "Technical Support Specialist"
    ],
    "Technical Support": [
      "Technical Support Specialist",
      "Customer Success Manager"
    ],
    "Supply Chain": [
      "Operations Manager"
    ],
    "Inventory Management": [
      "Operations Manager"
    ],
    "Process Improvement": [
      "Operations Manager",
      "Business Analyst"
    ],
    "Business Analysis": [
      "Business Analyst",
      "Product Manager"
    ],
    "Requirements Gathering": [
      "Business Analyst",
      "Product Manager"
    ],
    "Budget Management": [
      "Project Manager",
      "Operations Manager",
      "HR Generalist"
    ],
    "Mentoring": [
      "Engineering Manager",
      "Project Manager"
    ],
    "Interviewing": [
      "Recruiter",
      "HR Generalist"
    ]
  },
  "roleCatalog": [
    {
      "title": "Frontend Engineer",
      "seniority": "Mid?Senior",
      "summary": "Builds the interfaces people use every day, turning designs and product ideas into
fast, accessible …
      "skills": [
        "React",
        "TypeScript",
        "JavaScript",
        "HTML",
        "CSS",
        "Tailwind CSS",
        "Git",
        "Accessibility",
        "Testing"
      ]
    },
    {
      "title": "Backend Engineer",
      "seniority": "Mid?Senior",
      "summary": "Designs and ships the APIs, services, and data layers that power products behind the
scenes.",
      "skills": [
        "Node.js",
        "Express",
        "TypeScript",
        "MongoDB",
        "SQL",
        "REST APIs",
        "Docker",
        "Git",
        "System Design"
      ]
    },
    {
      "title": "Full-Stack Developer",
      "seniority": "Mid?Senior",
      "summary": "Works across the whole stack, from database schema to polished UI, owning features end
to end.",
      "skills": [
        "React",
        "Node.js",
        "TypeScript",
        "MongoDB",
```

```
        "REST APIs",
        "Git",
        "HTML",
        "CSS",
        "Testing"
      ]
    },
    {
      "title": "UI/UX Designer",
      "seniority": "Mid",
      "summary": "Researches, wireframes, and designs product experiences that are simple, consistent, and enjoyable t…
      "skills": [
        "Figma",
        "UI/UX Design",
        "Prototyping",
        "Wireframing",
        "Design Systems",
        "User Research",
        "Accessibility"
      ]
    },
    {
      "title": "Product Manager",
      "seniority": "Mid?Senior",
      "summary": "Decides what to build and why, balancing user needs, business goals, and engineering reality.",
      "skills": [
        "Roadmaps",
        "Product Strategy",
        "User Research",
        "Agile",
        "Stakeholder Management",
        "Communication",
        "A/B Testing"
      ]
    },
    {
      "title": "Data Analyst",
      "seniority": "Entry?Mid",
      "summary": "Turns raw data into clear answers, dashboards, and recommendations that teams act on.",
      "skills": [
        "SQL",
        "Python",
        "Excel",
        "Power BI",
        "Statistics",
        "Data Visualization",
        "Analytical Thinking"
      ]
    },
    {
      "title": "Data Scientist",
      "seniority": "Mid?Senior",
      "summary": "Builds models and experiments to predict outcomes and uncover patterns in complex data.",
      "skills": [
        "Python",
        "Machine Learning",
        "pandas",
        "Statistics",
        "SQL",
        "TensorFlow",
        "Data Visualization"
      ]
    },
    {
      "title": "Machine Learning Engineer",
      "seniority": "Mid?Senior",
      "summary": "Productionises AI models: training pipelines, evaluation, serving, and monitoring at scale.",
      "skills": [
        "Python",
        "PyTorch",
        "TensorFlow",
        "Machine Learning",
        "Docker",
        "AWS",
        "Kubernetes",
        "MLOps"
      ]
    },
    {
      "title": "DevOps Engineer",
      "seniority": "Mid?Senior",
      "summary": "Keeps delivery fast and reliable with CI/CD, infrastructure as code, and observability.",
      "skills": [
        "Docker",
```

```
        "Kubernetes",
        "CI/CD",
        "AWS",
        "Terraform",
        "Linux",
        "Monitoring",
        "Jenkins"
      ]
    },
    {
      "title": "Cloud Engineer",
      "seniority": "Mid?Senior",
      "summary": "Designs and runs cloud infrastructure that is scalable, secure, and cost-aware.",
      "skills": [
        "AWS",
        "Azure",
        "Google Cloud",
        "Kubernetes",
        "Terraform",
        "Networking",
        "Linux"
      ]
    },
    {
      "title": "Mobile App Developer",
      "seniority": "Mid",
      "summary": "Builds and ships mobile experiences for iOS and Android, native or cross-platform.",
      "skills": [
        "React Native",
        "Flutter",
        "Swift",
        "Kotlin",
        "JavaScript",
        "Testing",
        "Git"
      ]
    },
    {
      "title": "QA / Test Automation Engineer",
      "seniority": "Mid",
      "summary": "Protects product quality with test strategy, automated suites, and careful release
checks.",
      "skills": [
        "Testing",
        "Selenium",
        "Jest",
        "Cypress",
        "CI/CD",
        "Attention to Detail",
        "Quality Assurance"
      ]
    },
    {
      "title": "Cybersecurity Analyst",
      "seniority": "Mid",
      "summary": "Watches for threats, responds to incidents, and hardens systems and processes.",
      "skills": [
        "Security",
        "Networking",
        "Linux",
        "Incident Response",
        "Risk Management",
        "Monitoring"
      ]
    },
    {
      "title": "Database Administrator",
      "seniority": "Mid?Senior",
      "summary": "Owns data reliability: schema design, performance tuning, backups, and access control.",
      "skills": [
        "SQL",
        "PostgreSQL",
        "MySQL",
        "Performance Tuning",
        "Backup & Recovery",
        "Security"
      ]
    },
    {
      "title": "Engineering Manager",
      "seniority": "Senior",
      "summary": "Leads a team of engineers, unblocks delivery, and grows people through feedback and
mentoring.",
      "skills": [
        "Leadership",
        "Mentoring",
        "Agile",
        "System Design",
        "Stakeholder Management",
```

```
        "Communication"
      ]
    },
    {
      "title": "Technical Writer",
      "seniority": "Mid",
      "summary": "Makes complex products understandable through docs, guides, and clear examples.",
      "skills": [
        "Technical Writing",
        "Documentation",
        "HTML",
        "APIs",
        "Communication",
        "Editing"
      ]
    },
    {
      "title": "Digital Marketing Specialist",
      "seniority": "Entry?Mid",
      "summary": "Runs campaigns across search, social, and email, then measures what actually works.",
      "skills": [
        "SEO",
        "Content Marketing",
        "Google Analytics",
        "Social Media",
        "Copywriting",
        "Email Marketing"
      ]
    },
    {
      "title": "Content Strategist",
      "seniority": "Mid",
      "summary": "Plans what content to create, where it lives, and how it moves people through the
funnel.",
      "skills": [
        "Content Marketing",
        "Copywriting",
        "SEO",
        "User Research",
        "Communication",
        "Google Analytics"
      ]
    },
    {
      "title": "Growth Marketer",
      "seniority": "Mid?Senior",
      "summary": "Runs experiments across the funnel to acquire, activate, and retain more users.",
      "skills": [
        "A/B Testing",
        "Google Analytics",
        "SEO",
        "Email Marketing",
        "Analytics",
        "Experimentation"
      ]
    },
    {
      "title": "Sales Development Representative",
      "seniority": "Entry?Mid",
      "summary": "Builds pipeline by researching prospects, starting conversations, and qualifying
leads.",
      "skills": [
        "CRM",
        "Communication",
        "Negotiation",
        "Lead Generation",
        "Salesforce",
        "Time Management"
      ]
    },
    {
      "title": "Customer Success Manager",
      "seniority": "Mid",
      "summary": "Helps customers get value from a product, from onboarding to renewal.",
      "skills": [
        "Customer Success",
        "Communication",
        "CRM",
        "Onboarding",
        "Analytics",
        "Empathy"
      ]
    },
    {
      "title": "Recruiter / Talent Acquisition",
      "seniority": "Mid",
      "summary": "Finds and attracts talent, running sourcing, interviews, and offers end to end.",
      "skills": [
        "Recruitment",
```

```
        "Interviewing",
        "Communication",
        "CRM",
        "Employer Branding",
        "Stakeholder Management"
      ]
    },
    {
      "title": "Financial Analyst",
      "seniority": "Mid",
      "summary": "Analyses performance, builds models, and turns numbers into decisions.",
      "skills": [
        "Excel",
        "Financial Modeling",
        "Accounting",
        "Forecasting",
        "SQL",
        "Budgeting"
      ]
    },
    {
      "title": "Project Manager",
      "seniority": "Mid",
      "summary": "Keeps scope, schedule, and stakeholders aligned so teams deliver on time.",
      "skills": [
        "Agile",
        "Scrum",
        "Stakeholder Management",
        "Risk Management",
        "Budget Management",
        "Communication"
      ]
    },
    {
      "title": "HR Generalist",
      "seniority": "Entry?Mid",
      "summary": "Runs the people side of a company: hiring, onboarding, payroll, and policy.",
      "skills": [
        "Recruitment",
        "Payroll",
        "Onboarding",
        "Employment Law",
        "Communication",
        "HRIS"
      ]
    },
    {
      "title": "Video Editor",
      "seniority": "Entry?Mid",
      "summary": "Shapes raw footage into stories for social, product, and brand channels.",
      "skills": [
        "Premiere Pro",
        "After Effects",
        "Storytelling",
        "Color Grading",
        "Audio",
        "Creativity"
      ]
    },
    {
      "title": "Graphic Designer",
      "seniority": "Entry?Mid",
      "summary": "Creates visuals that carry a brand across web, print, and social.",
      "skills": [
        "Photoshop",
        "Illustrator",
        "Branding",
        "Typography",
        "Layout Design",
        "Canva"
      ]
    },
    {
      "title": "Operations Manager",
      "seniority": "Mid?Senior",
      "summary": "Runs the day-to-day engine of a business and keeps improving how work flows.",
      "skills": [
        "Process Improvement",
        "Supply Chain",
        "Excel",
        "Leadership",
        "Analytics",
        "Budget Management"
      ]
    },
    {
      "title": "Business Analyst",
      "seniority": "Mid",
      "summary": "Bridges business needs and technology by gathering requirements and shaping solutions.",
```

```
      "skills": [
        "Business Analysis",
        "Requirements Gathering",
        "SQL",
        "Power BI",
        "Process Improvement",
        "Communication"
      ]
    },
    {
      "title": "Technical Support Specialist",
      "seniority": "Entry?Mid",
      "summary": "Resolves customer and internal technical issues, and escalates what needs engineering.",
      "skills": [
        "Technical Support",
        "Customer Support",
        "Linux",
        "Networking",
        "Communication",
        "Problem Solving"
      ]
    }
  ]
}
```

package.json

```
{
  "name": "nextrole",
  "version": "1.0.0",
  "private": true,
  "description": "Next Role - Find work that fits. Upload a resume, get job roles matched to your skills.",
  "main": "electron/main.cjs",
  "engines": {
    "node": ">=20"
  },
  "scripts": {
    "postinstall": "npm install --prefix client && npm install --prefix server",
    "dev": "concurrently -k -n server,client -c green,cyan \"npm run dev --prefix server\" \"npm run dev --prefix clie…
    "build": "npm run build --prefix client && npm run build --prefix server",
    "typecheck": "npm run typecheck --prefix client && npm run typecheck --prefix server",
    "start": "npm run start --prefix server",
    "desktop": "electron .",
    "package:win": "electron-builder --win dir"
  },
  "devDependencies": {
    "concurrently": "^9.1.0",
    "electron": "^44.7.0",
    "electron-builder": "^26.15.3"
  },
  "build": {
    "appId": "com.nextrole.desktop",
    "productName": "Next Role",
    "asar": false,
    "directories": {
      "output": "release"
    },
    "files": [
      "electron/**",
      "server/dist/**",
      "server/node_modules/**",
      "client/dist/**",
      "nextrole.json",
      "package.json"
    ],
    "win": {
      "target": "dir"
    }
  }
}
```

README.md

```
# Next Role ? Find work that fits

Upload a resume, get the job roles that actually fit it ? with the exact skills
each role requires and a match score against your experience.

Built with **React + TypeScript + Tailwind CSS** on the front end and
```

Node + Express + MongoDB on the back end, with an offline matcher that can be upgraded to **Grok (xAI)** AI analysis and **Google OAuth** when keys are configured. Every piece of app data ? copy, skill catalogue, role library ? lives in a single file: [`nextrole.json`](./nextrole.json).

What it does

- **Reads any resume, even scans.** Three extraction engines run in order: pdf-parse ? pdf.js 3 ? **Tesseract OCR**. A photographed/scanned PDF with no text layer is rendered page-by-page and read with real text recognition, so "image-only" resumes still produce matches.
- **Smart matching without any API key.** Skill detection normalises real-world spellings (`ReactJS`, `Node JS`, `k8s`, `postgres`, `HTML5`...), reads title signals ("front end developer", "DevOps", "nurse"...) and experience length, then ranks the 30-role library. Grok upgrades the same flow to AI-generated matches when `GROK_API_KEY` is set.
- **Explore roles section.** Search the library, see your match % on every card (matched roles float to the top), and pick any role from the selector to see the full required-skills checklist ? including which skills you already have.
- **Three sections.** `Home` (the landing interface), `Explore roles` (the library), `Your profile` (saved analyses).
- **Secured accounts.** Email + password sign-up/sign-in (passwords hashed with scrypt), optional Google OAuth, JWT sessions. Works with MongoDB when configured and falls back to a file store inside `nextrole.json` ? no database needed to run.
- **Desktop app.** Electron wrapper that starts the bundled server and opens the app in its own window (see below).

Getting started

```bash
npm install          # installs root, client and server dependencies
cp .env.example .env # optional: Google / Grok / Mongo keys
npm run dev          # API on http://localhost:4000, UI on http://localhost:5173
```

Production (single server ? UI and API from one process):

```bash
npm run build
npm start            # http://localhost:4000 serves everything
```

Desktop app

```bash
npm run build        # once, to produce client/dist and server/dist
npm run desktop      # opens Next Role as a desktop app
```

Package a portable Windows build (folder with `Next Role.exe`, no installer, no signing needed):

```bash
npm run package:win  # output in release/
```

Environment variables

Everything is optional ? the app runs fully without them.

Variable	Purpose
`GOOGLE_CLIENT_ID` / `VITE_GOOGLE_CLIENT_ID`	Enables the Google sign-in button
`GROK_API_KEY`	AI-powered matching via xAI Grok (falls back to the local matcher)
`MONGODB_URI`	MongoDB persistence (falls back to the file store in `nextrole.json`)
`JWT_SECRET`	Session signing secret (a dev default is used otherwise)

Project structure

```
client/     React + Vite + Tailwind app (Home, Explore roles, Profile, Sign in/up)
server/     Express API: /api/analyze, /api/auth, /api/roles, /api/profile
electron/   Desktop app main process
nextrole.json  Single source of app data + optional runtime store
```

How matching works

1. **Extract** text (pdf.js engines, then OCR for scans).
2. **Detect skills** against the 163-entry catalogue in `nextrole.json`, with alias normalisation and word-boundary matching.
3. **Score roles** from the 30-role library: direct skill overlap, skills that imply a role, and title/experience signals from the resume itself.
4. Return a ranked list with per-role "why" explanations ? always at least six roles, best fit first.

When `GROK_API_KEY` is present, Grok performs steps 2?4 with the same catalogue as ground truth, and the local matcher remains as the fallback.

```
96|
```

server/package.json

```
 1| {
 2|   "name": "nextrole-server",
 3|   "version": "1.0.0",
 4|   "private": true,
 5|   "description": "Next Role API - resume analysis with Grok, MongoDB persistence, Google OAuth.",
 6|   "scripts": {
 7|     "dev": "tsx watch src/index.ts",
 8|     "build": "tsc -p tsconfig.json",
 9|     "start": "node dist/index.js",
10|     "typecheck": "tsc -p tsconfig.json --noEmit"
11|   },
12|   "dependencies": {
13|     "@tesseract.js-data/eng": "^1.0.0",
14|     "canvas": "^3.2.3",
15|     "cors": "^2.8.5",
16|     "dotenv": "^16.4.5",
17|     "express": "^4.21.2",
18|     "google-auth-library": "^9.14.0",
19|     "jsonwebtoken": "^9.0.2",
20|     "mammoth": "^1.8.0",
21|     "mongoose": "^8.7.0",
22|     "pdf-parse": "^1.1.1",
23|     "pdfjs-dist": "^3.11.174",
24|     "tesseract.js": "^7.0.0"
25|   },
26|   "devDependencies": {
27|     "@types/cors": "^2.8.17",
28|     "@types/express": "^4.17.21",
29|     "@types/jsonwebtoken": "^9.0.7",
30|     "@types/node": "^20.16.10",
31|     "tsx": "^4.19.0",
32|     "typescript": "^5.6.2"
33|   }
34| }
35|
```

server/src/data.ts

```
 1| import fs from 'node:fs';
 2| import path from 'node:path';
 3|
 4| export interface RoleDefinition {
 5|   title: string;
 6|   seniority: string;
 7|   summary: string;
 8|   skills: string[];
 9| }
10|
11| export interface NextRoleData {
12|   title: string;
13|   brand: { name: string; mark: string; tagline: string };
14|   experience: {
15|     errors: { empty: string; tooShort: string; server: string };
16|   };
17|   skillCatalog: string[];
18|   skillRoles: Record<string, string[]>;
19|   roleCatalog: RoleDefinition[];
20| }
21|
22| /**
23|  * All application data lives in a single file: `nextrole.json` at the project root.
24|  * It is read at runtime (dev and production builds resolve the same path).
25|  */
26| const candidates = [
27|   path.resolve(__dirname, '..', '..', 'nextrole.json'),
28|   path.resolve(process.cwd(), 'nextrole.json'),
29|   path.resolve(process.cwd(), '..', 'nextrole.json'),
30| ];
31|
32| function resolvePath(): string {
33|   for (const candidate of candidates) {
34|     if (fs.existsSync(candidate)) return candidate;
35|   }
36|   throw new Error('nextrole.json not found. Expected it at the project root.');
37| }
38|
39| /** Absolute path of the single data file. */
40| export const dataPath = resolvePath();
41|
```

```
export const data: NextRoleData = JSON.parse(fs.readFileSync(dataPath, 'utf-8')) as NextRoleData;

```

server/src/db.ts

```
import mongoose from 'mongoose';
import { env } from './env';

/** Connect to MongoDB without crashing the app when the database is unavailable. */
export async function connectDb(): Promise<void> {
  if (!env.mongoUri) {
    console.warn('[db] MONGODB_URI is not set - running without persistence.');
    return;
  }
  try {
    await mongoose.connect(env.mongoUri, { serverSelectionTimeoutMS: 4000 });
    console.log('[db] MongoDB connected');
  } catch (cause) {
    console.warn(
      '[db] MongoDB unavailable - continuing without persistence:',
      cause instanceof Error ? cause.message : cause,
    );
  }
}

export function isDbConnected(): boolean {
  return mongoose.connection.readyState === 1;
}

```

server/src/env.ts

```
import dotenv from 'dotenv';
import path from 'node:path';

// Load the shared .env from the project root, then any server-local overrides.
dotenv.config({ path: path.resolve(__dirname, '..', '..', '.env') });
dotenv.config();

export const env = {
  port: Number(process.env.PORT ?? 4000),
  mongoUri: process.env.MONGODB_URI ?? '',
  jwtSecret: process.env.JWT_SECRET ?? 'nextrole-dev-secret-change-me',
  clientOrigin: process.env.CLIENT_ORIGIN ?? 'http://localhost:5173',
  googleClientId: process.env.GOOGLE_CLIENT_ID ?? '',
  grokApiKey: process.env.GROK_API_KEY ?? '',
  grokModel: process.env.GROK_MODEL ?? 'grok-4-fast',
  grokApiUrl: process.env.GROK_API_URL ?? 'https://api.x.ai/v1/chat/completions',
};

```

server/src/index.ts

```
import cors from 'cors';
import express, { type NextFunction, type Request, type Response } from 'express';
import fs from 'node:fs';
import path from 'node:path';
import { data } from './data';
import { connectDb, isDbConnected } from './db';
import { env } from './env';
import { seedRoles } from './models';
import analyzeRouter from './routes/analyze';
import authRouter from './routes/auth';
import profileRouter from './routes/profile';
import rolesRouter from './routes/roles';

async function main(): Promise<void> {
  await connectDb();
  if (isDbConnected()) {
    await seedRoles(data.roleCatalog);
  }

  const app = express();

  app.use(cors({ origin: env.clientOrigin }));
  app.use(express.json({ limit: '1mb' }));

  app.get('/api/health', (_req, res) => {
    res.json({ ok: true, title: data.title, db: isDbConnected() });
  });
```

```

  app.use('/api/analyze', analyzeRouter);
  app.use('/api/auth', authRouter);
  app.use('/api/roles', rolesRouter);
  app.use('/api/profile', profileRouter);

  // Production / desktop mode: serve the built client from the same origin.
  const clientDist = path.resolve(__dirname, '..', '..', 'client', 'dist');
  const clientIndex = path.join(clientDist, 'index.html');
  if (fs.existsSync(clientIndex)) {
    app.use(
      express.static(clientDist, {
        setHeaders: (res, filePath) => {
          // The app shell must never be cached - stale copies break routing
          // (e.g. an old bundle without the /signup page). Hashed assets are
          // immutable, so they can stay cacheable.
          if (filePath.endsWith('.html')) res.setHeader('Cache-Control', 'no-store');
        },
      }),
    );
    // Single-page-app fallback: any non-API GET returns the client shell.
    app.get('*', (req, res, next) => {
      if (req.path.startsWith('/api')) {
        next();
        return;
      }
      res.setHeader('Cache-Control', 'no-store');
      res.sendFile(clientIndex);
    });
  }

  app.use((_req, res) => {
    res.status(404).json({ error: 'Not found.' });
  });

  // JSON errors (e.g. file larger than 10 MB).
  app.use((err: unknown, _req: Request, res: Response, next: NextFunction) => {
    if (res.headersSent) {
      next(err);
      return;
    }
    const type = (err as { type?: string } | null)?.type;
    if (type === 'entity.too.large') {
      res.status(413).json({ error: 'That file is larger than 10 MB.' });
      return;
    }
    console.error('[server] unhandled error:', err);
    res.status(500).json({ error: data.experience.errors.server });
  });

  app.listen(env.port, () => {
    console.log(`[server] Next Role API listening on http://localhost:${env.port}`);
    console.log(`[server] Grok AI ${env.grokApiKey ? 'enabled' : 'disabled (local matching)'}`);
  });
}

main().catch((cause) => {
  console.error('[server] failed to start:', cause);
  process.exit(1);
});

```

server/src/middleware/auth.ts

```

import type { NextFunction, Request, Response } from 'express';
import jwt from 'jsonwebtoken';
import { env } from '../env';

export type AuthedRequest = Request & { userId?: string };

export function signToken(userId: string): string {
  return jwt.sign({ sub: userId }, env.jwtSecret, { expiresIn: '7d' });
}

function readToken(req: Request): string | null {
  const header = req.headers.authorization;
  if (header && header.startsWith('Bearer ')) return header.slice(7).trim();
  return null;
}

/** Rejects requests without a valid session token. */
export function requireAuth(req: AuthedRequest, res: Response, next: NextFunction): void {
  const token = readToken(req);
  if (!token) {
    res.status(401).json({ error: 'Sign in required.' });
    return;
```

```
  }
  try {
    const payload = jwt.verify(token, env.jwtSecret) as jwt.JwtPayload;
    if (!payload.sub) throw new Error('Token missing subject');
    req.userId = payload.sub;
    next();
  } catch {
    res.status(401).json({ error: 'Your session expired. Please sign in again.' });
  }
}

/** Attaches userId when a valid token is present, otherwise continues. */
export function optionalAuth(req: AuthedRequest, res: Response, next: NextFunction): void {
  const token = readToken(req);
  if (token) {
    try {
      const payload = jwt.verify(token, env.jwtSecret) as jwt.JwtPayload;
      if (payload.sub) req.userId = payload.sub;
    } catch {
      // Ignore invalid tokens on public routes.
    }
  }
  next();
}

export function isObjectIdString(value: string): boolean {
  return /^[a-f\d]{24}$/i.test(value);
}

```

server/src/models.ts

```
import { Schema, model, type Types } from 'mongoose';
import type { AnalysisSource, RoleMatch } from './types';

export interface UserDoc {
  _id: Types.ObjectId;
  googleId: string;
  name: string;
  email: string;
  picture?: string;
  passwordHash?: string;
}

const userSchema = new Schema<UserDoc>(
  {
    googleId: { type: String, required: true, index: true },
    name: { type: String, required: true },
    email: { type: String, required: true, index: true },
    picture: { type: String },
    passwordHash: { type: String },
  },
  { timestamps: true },
);

export const User = model<UserDoc>('User', userSchema);

export interface RoleDoc {
  title: string;
  seniority: string;
  summary: string;
  skills: string[];
}

const roleSchema = new Schema<RoleDoc>(
  {
    title: { type: String, required: true, unique: true },
    seniority: { type: String, required: true },
    summary: { type: String, required: true },
    skills: { type: [String], default: [] },
  },
  { timestamps: true },
);

export const Role = model<RoleDoc>('Role', roleSchema);

export interface AnalysisDoc {
  _id: Types.ObjectId;
  user?: Types.ObjectId;
  fileName?: string;
  skills: string[];
  summary: string;
  roles: RoleMatch[];
  source: AnalysisSource;
  createdAt?: Date;
  updatedAt?: Date;
```

```
}

const roleMatchSchema = new Schema<RoleMatch>(
  {
    title: { type: String, required: true },
    matchPercent: { type: Number, required: true },
    seniority: { type: String, default: '' },
    summary: { type: String, default: '' },
    why: { type: String, default: '' },
    skills: { type: [String], default: [] },
  },
  { _id: false },
);

const analysisSchema = new Schema<AnalysisDoc>(
  {
    user: { type: Schema.Types.ObjectId, ref: 'User', index: true },
    fileName: { type: String },
    skills: { type: [String], default: [] },
    summary: { type: String, default: '' },
    roles: { type: [roleMatchSchema], default: [] },
    source: { type: String, enum: ['grok', 'local'], default: 'local' },
  },
  { timestamps: true },
);

export const Analysis = model<AnalysisDoc>('Analysis', analysisSchema);

/** Idempotent seed of the role library from nextrole.json. */
export async function seedRoles(
  catalog: Array<{ title: string; seniority: string; summary: string; skills: string[] }>,
): Promise<void> {
  for (const role of catalog) {
    try {
      await Role.updateOne({ title: role.title }, { $setOnInsert: role }, { upsert: true });
    } catch {
      // Database offline - the API falls back to the JSON catalog.
      return;
    }
  }
}

```

server/src/routes/analyze.ts

```
import { Router, json, raw, type Response } from 'express';
import { Types } from 'mongoose';
import { data } from '../data';
import { isDbConnected } from '../db';
import { type AuthedRequest, optionalAuth } from '../middleware/auth';
import { Analysis } from '../models';
import { cleanText, ExtractionError, extractText } from '../services/extract';
import { analyzeWithGrok } from '../services/grok';
import { matchLocally } from '../services/matcher';
import * as fileStore from '../store';

const router = Router();

router.use(raw({ type: 'application/octet-stream', limit: '10mb' }));

router.post('/', optionalAuth, async (req: AuthedRequest, res: Response) => {
  try {
    const contentType = String(req.headers['content-type'] ?? '');
    let text = '';
    let fileName: string | undefined;

    if (contentType.includes('application/json')) {
      const body = (req.body ?? {}) as { text?: unknown };
      text = typeof body.text === 'string' ? body.text : '';
    } else if (Buffer.isBuffer(req.body)) {
      const header = req.headers['x-filename'];
      fileName = typeof header === 'string' ? decodeURIComponent(header) : 'resume.txt';
      text = await extractText(req.body, fileName);
    } else {
      res.status(415).json({ error: 'Unsupported request. Send JSON text or a raw file.' });
      return;
    }

    text = cleanText(text);

    if (!text) {
      res.status(400).json({ error: data.experience.errors.empty });
      return;
    }
    if (text.length < 60) {
      res.status(400).json({ error: data.experience.errors.tooShort });
```

```
  42|       return;
  43|     }
  44|
  45|     // Prefer Grok, fall back to the local matcher from nextrole.json.
  46|     const grokResult = await analyzeWithGrok(text);
  47|     const analysis = grokResult ?? matchLocally(text);
  48|     const source: 'grok' | 'local' = grokResult ? 'grok' : 'local';
  49|     console.log(
  50|       `[analyze] ${fileName ?? 'pasted text'}: ${text.length} chars, ${analysis.skills.length} skills,
${analysis.role…
  51|     );
  52|
  53|     let saved = false;
  54|     if (req.userId) {
  55|       if (isDbConnected() && Types.ObjectId.isValid(req.userId)) {
  56|         try {
  57|           await Analysis.create({
  58|             user: new Types.ObjectId(req.userId),
  59|             fileName,
  60|             skills: analysis.skills,
  61|             summary: analysis.summary,
  62|             roles: analysis.roles,
  63|             source,
  64|           });
  65|           saved = true;
  66|         } catch (cause) {
  67|           console.warn('[analyze] could not save analysis:', cause);
  68|         }
  69|       } else if (!isDbConnected()) {
  70|         // No MongoDB: keep the user's history in the nextrole.json store.
  71|         try {
  72|           fileStore.saveAnalysis(req.userId, {
  73|             fileName,
  74|             skills: analysis.skills,
  75|             summary: analysis.summary,
  76|             roles: analysis.roles,
  77|             source,
  78|           });
  79|           saved = true;
  80|         } catch (cause) {
  81|           console.warn('[analyze] could not save analysis to file store:', cause);
  82|         }
  83|       }
  84|     }
  85|
  86|     res.json({ ok: true, saved, fileName, source, ...analysis });
  87|   } catch (cause) {
  88|     if (cause instanceof ExtractionError) {
  89|       res.status(422).json({ error: cause.message });
  90|       return;
  91|     }
  92|     console.error('[analyze] failed:', cause);
  93|     res.status(500).json({ error: data.experience.errors.server });
  94|   }
  95| });
  96|
  97| export default router;
  98|
```

server/src/routes/auth.ts

```
   1| import { Router, type Response } from 'express';
   2| import { OAuth2Client } from 'google-auth-library';
   3| import { env } from '../env';
   4| import { isDbConnected } from '../db';
   5| import { type AuthedRequest, signToken } from '../middleware/auth';
   6| import { User } from '../models';
   7| import { hashPassword, verifyPassword } from '../services/passwords';
   8| import * as fileStore from '../store';
   9|
  10| const router = Router();
  11|
  12| const EMAIL_PATTERN = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  13| const MIN_PASSWORD_LENGTH = 6;
  14|
  15| type AccountRecord = {
  16|   source: 'mongo' | 'store';
  17|   id: string;
  18|   name: string;
  19|   email: string;
  20|   picture?: string;
  21|   passwordHash?: string;
  22| };
  23|
  24| /** Loads an account by email from MongoDB (when connected) or the file store. */
  25| async function findAccount(email: string): Promise<AccountRecord | null> {
```

```
  if (isDbConnected()) {
    try {
      const user = await User.findOne({ email });
      if (user) {
        return {
          source: 'mongo',
          id: user._id.toString(),
          name: user.name,
          email: user.email,
          picture: user.picture,
          passwordHash: user.passwordHash,
        };
      }
    } catch (cause) {
      console.warn('[auth] could not read user from MongoDB:', cause);
    }
  }
  const stored = fileStore.findUserByEmail(email);
  if (stored) {
    return {
      source: 'store',
      id: stored.id,
      name: stored.name,
      email: stored.email,
      picture: stored.picture,
      passwordHash: stored.passwordHash,
    };
  }
  return null;
}

/** Persists a password hash onto an existing account. */
async function setAccountPassword(account: AccountRecord, passwordHash: string): Promise<void> {
  if (account.source === 'mongo') {
    try {
      await User.updateOne({ _id: account.id }, { $set: { passwordHash } });
      return;
    } catch (cause) {
      console.warn('[auth] could not save password to MongoDB:', cause);
    }
  }
  fileStore.setUserPasswordHash(account.id, passwordHash);
}

/**
 * Upserts a user in MongoDB when connected, otherwise in the file store
 * inside `nextrole.json`. Always returns a stable userId.
 */
async function startSession(
  profile: {
    googleId?: string;
    name: string;
    email: string;
    picture?: string;
    passwordHash?: string;
  },
  res: Response,
): Promise<void> {
  let userId: string | undefined;
  let mongoId: string | undefined;

  if (isDbConnected()) {
    try {
      const user = await User.findOneAndUpdate(
        {
          $or: [
            ...(profile.googleId ? [{ googleId: profile.googleId }] : []),
            { email: profile.email },
          ],
        },
        {
          $set: {
            ...(profile.googleId ? { googleId: profile.googleId } : {}),
            name: profile.name,
            email: profile.email,
            ...(profile.picture ? { picture: profile.picture } : {}),
            ...(profile.passwordHash ? { passwordHash: profile.passwordHash } : {}),
          },
          $setOnInsert: { googleId: profile.googleId ?? `local:${profile.email.toLowerCase()}` },
        },
        { new: true, upsert: true, setDefaultsOnInsert: true },
      );
      mongoId = user?._id.toString();
      userId = mongoId;
    } catch (cause) {
      console.warn('[auth] could not persist user to MongoDB:', cause);
    }
  }

```

```
  if (!userId) {
    const stored = fileStore.upsertUser(profile);
    userId = stored.id;
  }

  res.json({
    token: signToken(userId),
    user: {
      id: userId,
      name: profile.name,
      email: profile.email,
      picture: profile.picture,
    },
  });
}

/** Verifies a Google ID token (Google OAuth) and starts a Next Role session. */
router.post('/google', async (req: AuthedRequest, res: Response) => {
  const credential = (req.body ?? {}) as { credential?: unknown };
  if (typeof credential.credential !== 'string' || !credential.credential.trim()) {
    res.status(400).json({ error: 'Missing Google credential.' });
    return;
  }

  try {
    const client = new OAuth2Client();
    const ticket = await client.verifyIdToken({
      idToken: credential.credential,
      ...(env.googleClientId ? { audience: env.googleClientId } : {}),
    });
    const payload = ticket.getPayload();

    if (!payload?.sub || !payload.email) {
      res.status(401).json({ error: 'Google sign-in could not be verified.' });
      return;
    }

    await startSession(
      {
        googleId: payload.sub,
        name: payload.name ?? payload.email,
        email: payload.email,
        picture: payload.picture,
      },
      res,
    );
  } catch (cause) {
    console.warn('[auth] google verification failed:', cause);
    res.status(401).json({ error: 'Google sign-in failed. Please try again.' });
  }
});

/**
 * Password sign-in and sign-up.
 *   POST /api/auth/local { mode: 'signup', name, email, password }  -> create account
 *   POST /api/auth/local { mode: 'signin', email, password }        -> sign in
 * Works with no external configuration, so the flow is always usable
 * alongside Google OAuth.
 */
router.post('/local', async (req: AuthedRequest, res: Response) => {
  const body = (req.body ?? {}) as {
    name?: unknown;
    email?: unknown;
    password?: unknown;
    mode?: unknown;
  };
  const mode = body.mode === 'signup' ? 'signup' : 'signin';
  const name = typeof body.name === 'string' ? body.name.trim() : '';
  const email = (typeof body.email === 'string' ? body.email.trim() : '').toLowerCase();
  const password = typeof body.password === 'string' ? body.password : '';

  if (!EMAIL_PATTERN.test(email)) {
    res.status(400).json({ error: 'Please enter a valid email address.' });
    return;
  }
  if (password.length < MIN_PASSWORD_LENGTH) {
    res
      .status(400)
      .json({ error: `Your password must be at least ${MIN_PASSWORD_LENGTH} characters long.` });
    return;
  }

  try {
    if (mode === 'signup') {
      if (name.length < 2) {
        res.status(400).json({ error: 'Please enter your name.' });
        return;
      }
      const existing = await findAccount(email);
```

```
      if (existing) {
        res
          .status(409)
          .json({ error: 'An account with this email already exists. Please sign in instead.' });
        return;
      }
      await startSession({ name, email, passwordHash: hashPassword(password) }, res);
      return;
    }

    // ---- sign in ----
    const account = await findAccount(email);
    if (!account) {
      res.status(404).json({ error: 'No account found with this email. Please sign up first.' });
      return;
    }
    if (!account.passwordHash) {
      // Legacy passwordless account (created before passwords): secure it now.
      await setAccountPassword(account, hashPassword(password));
    } else if (!verifyPassword(password, account.passwordHash)) {
      res.status(401).json({ error: 'Incorrect email or password. Please try again.' });
      return;
    }
    await startSession(
      { name: account.name, email: account.email, picture: account.picture },
      res,
    );
  } catch (cause) {
    console.warn('[auth] local sign-in failed:', cause);
    res.status(500).json({ error: 'Sign in failed. Please try again.' });
  }
});

export default router;
```

server/src/routes/profile.ts

```
import { Router, type Response } from 'express';
import { Types } from 'mongoose';
import { isDbConnected } from '../db';
import { type AuthedRequest, isObjectIdString, requireAuth } from '../middleware/auth';
import { Analysis } from '../models';
import * as fileStore from '../store';

const router = Router();

router.use(requireAuth);

/** Saved analyses for the signed-in user, newest first. */
router.get('/', async (req: AuthedRequest, res: Response) => {
  const userId = req.userId ?? '';

  // MongoDB when connected, otherwise the store inside nextrole.json.
  if (isDbConnected() && isObjectIdString(userId)) {
    try {
      const docs = await Analysis.find({ user: new Types.ObjectId(userId) })
        .sort({ createdAt: -1 })
        .limit(30)
        .lean();

      res.json({
        count: docs.length,
        analyses: docs.map((doc) => ({
          id: doc._id.toString(),
          fileName: doc.fileName,
          summary: doc.summary,
          skills: doc.skills ?? [],
          roles: doc.roles ?? [],
          source: doc.source,
          createdAt: doc.createdAt ?? new Date(),
        })),
      });
      return;
    } catch (cause) {
      console.warn('[profile] could not load analyses:', cause);
    }
  }

  if (isDbConnected() || !userId) {
    res.json({ count: 0, analyses: [] });
    return;
  }

  const analyses = fileStore.listAnalyses(userId);
  res.json({ count: analyses.length, analyses });
```

```
});

/** Delete one saved analysis. */
router.delete('/analysis/:id', async (req: AuthedRequest, res: Response) => {
  const userId = req.userId ?? '';
  const id = req.params.id ?? '';

  if (isDbConnected() && isObjectIdString(userId) && Types.ObjectId.isValid(id)) {
    try {
      const result = await Analysis.deleteOne({
        _id: new Types.ObjectId(id),
        user: new Types.ObjectId(userId),
      });
      if (result.deletedCount === 0) {
        res.status(404).json({ error: 'Analysis not found.' });
        return;
      }
      res.json({ ok: true });
      return;
    } catch (cause) {
      console.warn('[profile] delete failed:', cause);
      res.status(500).json({ error: 'Could not delete that analysis.' });
      return;
    }
  }

  if (!isDbConnected() && fileStore.deleteAnalysis(userId, id)) {
    res.json({ ok: true });
    return;
  }

  res.status(404).json({ error: 'Analysis not found.' });
});

export default router;
```

server/src/routes/roles.ts

```
import { Router, type Response } from 'express';
import { data } from '../data';
import { isDbConnected } from '../db';
import { Role } from '../models';

const router = Router();

/** Role library - served from MongoDB when available, otherwise from nextrole.json. */
router.get('/', async (_req, res: Response) => {
  if (isDbConnected()) {
    try {
      const roles = await Role.find().lean();
      if (roles.length > 0) {
        res.json({ roles });
        return;
      }
    } catch (cause) {
      console.warn('[roles] falling back to nextrole.json:', cause);
    }
  }
  res.json({ roles: data.roleCatalog });
});

export default router;
```

server/src/services/extract.ts

```
import path from 'node:path';
import { createRequire } from 'node:module';

const requireCjs = createRequire(__filename);

type PdfParseResult = { text?: string; numpages?: number };
type MammothModule = {
  extractRawText(input: { buffer: Buffer }): Promise<{ value: string; messages: unknown[] }>;
};

/** Minimal structural types for the pdf.js legacy (CJS) build. */
type PdfjsTextContent = { items: Array<{ str?: unknown }> };
type PdfjsPage = { getTextContent(): Promise<PdfjsTextContent> };
type PdfjsDocument = {
  numPages: number;
  getPage(n: number): Promise<PdfjsPage>;
```

```
  destroy(): Promise<void>;
};
type PdfjsModule = {
  getDocument(params: Record<string, unknown>): { promise: Promise<PdfjsDocument> };
};

// Required from the package internals so pdf-parse never enters its debug mode.
const pdfParse = requireCjs('pdf-parse/lib/pdf-parse.js') as (
  buffer: Uint8Array,
) => Promise<PdfParseResult>;
const mammoth = requireCjs('mammoth') as MammothModule;
// Third engine: OCR for scanned/image-only PDFs (lazy - only loads on demand).
const ocrPdf = (): ((bytes: Uint8Array) => Promise<string>) =>
  requireCjs('./ocr').ocrPdf as (bytes: Uint8Array) => Promise<string>;

export class ExtractionError extends Error {
  constructor(message: string) {
    super(message);
    this.name = 'ExtractionError';
  }
}

/**
 * Body-parser delivers uploaded bodies as *views* into Node's pooled
 * ArrayBuffer (e.g. 64 KB of shared memory around a 2 KB PDF). Copy such views
 * into a tightly-sized buffer first.
 */
function detach(input: Buffer): Buffer {
  if (input.byteOffset === 0 && input.length === input.buffer.byteLength) return input;
  const out = Buffer.alloc(input.length); // exact-size, non-pooled ArrayBuffer
  input.copy(out);
  return out;
}

/**
 * Second opinion for PDFs the bundled pdf.js 1.x (2018) engine trips over:
 * modern layouts, unusual encodings, two-column exports. pdf.js 3.x is far
 * more tolerant, so we take whichever engine reads more text.
 */
async function extractWithPdfjs(bytes: Uint8Array): Promise<string> {
  const pdfjs = requireCjs('pdfjs-dist/legacy/build/pdf.js') as PdfjsModule;
  const doc = await pdfjs.getDocument({
    data: bytes,
    isEvalSupported: false,
    disableFontFace: true,
    useWorkerFetch: false,
    verbosity: 0,
  }).promise;

  try {
    const pages: string[] = [];
    for (let pageNumber = 1; pageNumber <= doc.numPages; pageNumber++) {
      const page = await doc.getPage(pageNumber);
      const content = await page.getTextContent();
      const text = content.items
        .map((item) => (typeof item.str === 'string' ? item.str : ''))
        .join(' ')
        .trim();
      if (text) pages.push(text);
    }
    return pages.join('\n');
  } finally {
    await doc.destroy().catch(() => undefined);
  }
}

async function extractPdf(bytes: Uint8Array, fileName: string): Promise<string> {
  let text = '';
  let primaryError: unknown;

  // Engine 1: pdf-parse (fast, covers most resumes).
  try {
    const result = await pdfParse(bytes);
    text = (result.text ?? '').trim();
  } catch (cause) {
    primaryError = cause;
  }

  // Engine 2: pdf.js 3.x when engine 1 failed or read almost nothing.
  if (text.length < 40) {
    try {
      const alt = await extractWithPdfjs(bytes);
      if (alt.trim().length > text.length) text = alt.trim();
    } catch (cause) {
      console.warn(
        '[extract] pdfjs fallback failed:',
        cause instanceof Error ? cause.message : cause,
      );
    }
```

```
  }

  // Engine 3: OCR for scanned/image-only PDFs - there is no text layer at
  // all, so the only way to read them is to look at the rendered page.
  if (text.length < 40) {
    try {
      console.log(`[extract] no text layer in ${fileName}; running OCR...`);
      const alt = await ocrPdf()(bytes);
      if (alt.trim().length > text.length) text = alt.trim();
      console.log(`[extract] OCR read ${text.length} chars from ${fileName}`);
    } catch (cause) {
      console.warn('[extract] ocr failed:', cause instanceof Error ? cause.message : cause);
    }
  }

  if (!text) {
    if (primaryError) {
      console.warn(
        '[extract] pdf-parse failed:',
        primaryError instanceof Error ? primaryError.message : primaryError,
      );
    }
    console.warn(`[extract] no text extracted from ${fileName} (0 chars)`);
    throw new ExtractionError(
      'We could not read this PDF, even with OCR. The file may be damaged or password-protected - try re-
exporting it …
    );
  }

  return text;
}

const TEXT_EXTENSIONS = ['.txt', '.text', '.md', '.csv', '.rtf'];

/** Extracts plain text from a PDF, DOCX, or plain-text resume. */
export async function extractText(buffer: Buffer, fileName: string): Promise<string> {
  const bytes = detach(buffer);
  const ext = path.extname(fileName ?? '').toLowerCase();

  if (ext === '.pdf') {
    // pdf.js (bundled inside pdf-parse) re-copies incoming data through the
    // deprecated `new Buffer(value)` constructor, which lands in Node's shared
    // memory pool with a garbage tail it then parses - breaking whenever other
    // modules (mongoose/bson) have dirtied the pool. A plain Uint8Array clones
    // to an exact-size copy instead, which parses reliably everywhere.
    const pdfBytes = new Uint8Array(bytes);
    return extractPdf(pdfBytes, fileName);
  }

  if (ext === '.docx') {
    const result = await mammoth.extractRawText({ buffer: bytes });
    const text = (result.value ?? '').trim();
    if (!text) {
      throw new ExtractionError(
        'That Word document appears to be empty or image-based. Paste your resume text instead.',
      );
    }
    return text;
  }

  if (ext === '.doc') {
    throw new ExtractionError(
      'Legacy .doc files are not supported. Re-save your resume as .docx, PDF, or .txt and try again.',
    );
  }

  if (ext && !TEXT_EXTENSIONS.includes(ext) && ext !== '') {
    throw new ExtractionError(
      `We could not read "${ext}" files. Please upload a PDF, Word document, or plain text file.`,
    );
  }

  return bytes.toString('utf-8');
}

/** Normalises extracted resume text. */
export function cleanText(raw: string): string {
  return raw
    .replace(/\u0000/g, '')
    .replace(/\r\n/g, '\n')
    .replace(/[ \t]+/g, ' ')
    .replace(/\n{3,}/g, '\n\n')
    .trim();
}

```

server/src/services/grok.ts

```ts
import { data } from '../data';
import { env } from '../env';
import type { AnalysisResult, RoleMatch } from '../types';

const MAX_INPUT_CHARS = 14_000;

function buildSystemPrompt(): string {
  const skills = data.skillCatalog.join(', ');
  const roles = data.roleCatalog.map((role) => role.title).join(', ');

  return `You are "Next Role", a precise career-matching engine.
Read the resume the user sends and return STRICT JSON only - no markdown fences, no commentary.

JSON shape:
{
  "skills": string[],
  "summary": string,
  "roles": [
    {
      "title": string,
      "seniority": string,
      "matchPercent": number,
      "summary": string,
      "why": string,
      "skills": string[]
    }
  ]
}

Rules:
- "skills": 6-14 real skills evidenced in the resume. Use these exact spellings when possible: ${skills}
- "summary": one line, max 220 characters, describing the person's profile in plain language.
- "roles": 5 to 7 roles, best match first. Prefer titles from this list: ${roles}
- "seniority": one of "Entry", "Entry-Mid", "Mid", "Mid-Senior", "Senior".
- "matchPercent": integer 50-98, an honest fit for THIS person.
- "summary" of a role: max 180 characters describing the role.
- "why": max 220 characters, citing concrete evidence from this resume.
- "skills": 3-6 of the person's skills that the role needs.
- Never invent employers, dates, degrees, or years of experience.
- If the resume is not a career document, still return valid JSON with the closest roles.`;
}

function toStrings(value: unknown): string[] {
  if (!Array.isArray(value)) return [];
  return value
    .filter((item): item is string => typeof item === 'string')
    .map((item) => item.trim())
    .filter(Boolean);
}

function asText(value: unknown, fallback = ''): string {
  return typeof value === 'string' ? value.trim() : fallback;
}

function normalizeRole(value: unknown): RoleMatch | null {
  if (!value || typeof value !== 'object') return null;
  const source = value as Record<string, unknown>;
  const title = asText(source.title);
  if (!title) return null;

  const raw = Number(source.matchPercent);
  const matchPercent = Number.isFinite(raw)
    ? Math.min(98, Math.max(40, Math.round(raw)))
    : 60;

  return {
    title: title.slice(0, 90),
    matchPercent,
    seniority: asText(source.seniority, 'Mid').slice(0, 30),
    summary: asText(source.summary).slice(0, 220),
    why: asText(source.why).slice(0, 320),
    skills: toStrings(source.skills).slice(0, 6),
  };
}

function parseGrokContent(raw: string): AnalysisResult | null {
  if (!raw) return null;
  const stripped = raw.replace(/```[a-z]*/gi, '').trim();
  const start = stripped.indexOf('{');
  const end = stripped.lastIndexOf('}');
  const jsonText = start >= 0 && end > start ? stripped.slice(start, end + 1) : stripped;

  try {
    const parsed = JSON.parse(jsonText) as Partial<AnalysisResult>;
    const roles = (Array.isArray(parsed.roles) ? parsed.roles : [])
```

```
      .map(normalizeRole)
      .filter((role): role is RoleMatch => role !== null)
      .sort((a, b) => b.matchPercent - a.matchPercent)
      .slice(0, 8);

    if (roles.length === 0) return null;

    return {
      skills: toStrings(parsed.skills).slice(0, 16),
      summary: asText(parsed.summary).slice(0, 300),
      roles,
    };
  } catch {
    return null;
  }
}

/**
 * Calls the xAI Grok API (OpenAI-compatible chat completions).
 * Returns null when unconfigured, rate-limited, or malformed so callers can fall back.
 */
export async function analyzeWithGrok(resumeText: string): Promise<AnalysisResult | null> {
  if (!env.grokApiKey) return null;

  try {
    const response = await fetch(env.grokApiUrl, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        Authorization: `Bearer ${env.grokApiKey}`,
      },
      body: JSON.stringify({
        model: env.grokModel,
        temperature: 0.3,
        response_format: { type: 'json_object' },
        messages: [
          { role: 'system', content: buildSystemPrompt() },
          { role: 'user', content: `RESUME:\n${resumeText.slice(0, MAX_INPUT_CHARS)}` },
        ],
      }),
      signal: AbortSignal.timeout(45_000),
    });

    if (!response.ok) {
      const detail = await response.text().catch(() => '');
      console.warn(`[grok] ${response.status}: ${detail.slice(0, 300)}`);
      return null;
    }

    const body = (await response.json()) as {
      choices?: Array<{ message?: { content?: string } }>;
    };
    return parseGrokContent(body.choices?.[0]?.message?.content ?? '');
  } catch (cause) {
    console.warn('[grok] request failed:', cause instanceof Error ? cause.message : cause);
    return null;
  }
}

```

server/src/services/matcher.ts

```
import { data, type RoleDefinition } from '../data';
import type { AnalysisResult, RoleMatch } from '../types';

const REGEX_SPECIALS = /[.*+?^${}()|[\]\\]/g;

function escapeRegExp(value: string): string {
  return value.replace(REGEX_SPECIALS, '\\$&');
}

/**
 * Word-boundary phrase match. Separators (space, -, /) are interchangeable so
 * "CI/CD", "ci-cd", "cicd" and "CI CD" all match the same skill.
 */
function containsPhrase(haystack: string, needle: string): boolean {
  const body = escapeRegExp(needle.toLowerCase()).replace(/[\s/-]+/g, '[\\s/-]*');
  const pattern = new RegExp(`(?<![a-z0-9])${body}(?![a-z0-9])`, 'i');
  return pattern.test(haystack);
}

/**
 * Real-world spellings mapped to their canonical skillCatalog entry.
 * Resumes rarely use catalog-perfect names ("reactjs", "k8s", "postgres"...).
 */
const SKILL_ALIASES: Record<string, string> = {
```

```
  js: 'JavaScript', es6: 'JavaScript', ecmascript: 'JavaScript',
  ts: 'TypeScript',
  reactjs: 'React', 'react js': 'React', redux: 'React',
  nextjs: 'Next.js', 'next js': 'Next.js',
  vue: 'Vue.js', vuejs: 'Vue.js', 'vue js': 'Vue.js',
  angularjs: 'Angular',
  nodejs: 'Node.js', 'node js': 'Node.js',
  expressjs: 'Express', 'express js': 'Express',
  spring: 'Spring Boot', 'springframework': 'Spring Boot',
  laravel: 'PHP', rails: 'Ruby', 'ruby on rails': 'Ruby',
  mongo: 'MongoDB',
  postgres: 'PostgreSQL', psql: 'PostgreSQL',
  mariadb: 'MySQL',
  tailwind: 'Tailwind CSS', tailwindcss: 'Tailwind CSS',
  rest: 'REST APIs', 'rest api': 'REST APIs', 'rest apis': 'REST APIs', restful: 'REST APIs',
  cicd: 'CI/CD', 'ci-cd': 'CI/CD', 'ci cd': 'CI/CD', 'continuous integration': 'CI/CD',
  k8s: 'Kubernetes',
  gcp: 'Google Cloud', 'google cloud platform': 'Google Cloud',
  'amazon web services': 'AWS', ec2: 'AWS', s3: 'AWS', cloudformation: 'AWS',
  'azure devops': 'Azure',
  ml: 'Machine Learning', llm: 'Machine Learning',
  'generative ai': 'Prompt Engineering', genai: 'Prompt Engineering',
  chatgpt: 'Prompt Engineering', 'prompt engineering': 'Prompt Engineering',
  sklearn: 'scikit-learn', 'scikit learn': 'scikit-learn',
  keras: 'TensorFlow', jupyter: 'Python',
  opencv: 'Computer Vision',
  powerbi: 'Power BI', 'microsoft excel': 'Excel', 'ms excel': 'Excel', vlookup: 'Excel',
  'google sheets': 'Google Sheets',
  ga4: 'Google Analytics', 'google analytics 4': 'Google Analytics',
  html5: 'HTML', css3: 'CSS',
  ux: 'UI/UX Design', 'ux design': 'UI/UX Design', 'ui design': 'UI/UX Design',
  wireframe: 'Wireframing', wireframes: 'Wireframing',
  prototype: 'Prototyping', prototypes: 'Prototyping',
  'design system': 'Design Systems',
  wcag: 'Accessibility',
  roadmap: 'Roadmaps',
  'product mgmt': 'Product Management',
  'team work': 'Teamwork',
  'verbal communication': 'Communication', 'written communication': 'Communication',
  'financial modelling': 'Financial Modeling',
  'risk analysis': 'Risk Management',
  'customer service': 'Customer Support',
  'tech support': 'Technical Support', helpdesk: 'Technical Support', 'help desk': 'Technical Support',
  'talent acquisition': 'Recruitment', recruiting: 'Recruitment',
  'statistical analysis': 'Statistics',
  'data viz': 'Data Visualization',
  snowflake: 'Data Warehousing', redshift: 'Data Warehousing', bigquery: 'Data Warehousing',
  grafana: 'Monitoring', prometheus: 'Monitoring', datadog: 'Monitoring', splunk: 'Monitoring',
  lambda: 'Serverless',
  'material-ui': 'React',
  'next js app': 'Next.js',
  'microsoft word': 'Word', 'ms word': 'Word',
  'ms powerpoint': 'PowerPoint', 'microsoft powerpoint': 'PowerPoint',
  'motion graphics': 'After Effects', 'video editing': 'Premiere Pro', premiere: 'Premiere Pro',
  'content writer': 'Copywriting', 'content writing': 'Copywriting',
  confluence: 'Documentation',
  'unit tests': 'Unit Testing',
  'distributed systems': 'System Design',
  'problem-solving': 'Problem Solving',
  'business-process-improvement': 'Process Improvement',
};

const catalogue = new Set(data.skillCatalog.map((skill) => skill.toLowerCase()));

/** Every catalog skill present in the resume, including alias spellings. */
function findSkills(text: string): string[] {
  const found: string[] = [];
  const seen = new Set<string>();

  const add = (skill: string) => {
    const key = skill.toLowerCase();
    if (seen.has(key)) return;
    seen.add(key);
    found.push(skill);
  };

  for (const skill of data.skillCatalog) {
    if (containsPhrase(text, skill)) add(skill);
  }

  for (const [alias, canonical] of Object.entries(SKILL_ALIASES)) {
    if (!catalogue.has(canonical.toLowerCase())) continue;
    if (seen.has(canonical.toLowerCase())) continue;
    if (containsPhrase(text, alias)) add(canonical);
  }

  return found;
}

```

```
/** Resume phrases that point strongly at a specific role in the library. */
const TITLE_HINTS: { pattern: RegExp; role: string; label: string }[] = [
  { pattern: /\b(front[\s-]?end|frontend|web developer|webdev|web designer)\b/i, role: 'Frontend Engineer', label: 'fr…
  { pattern: /\b(back[\s-]?end|backend|api developer|server[\s-]?side)\b/i, role: 'Backend Engineer', label: 'back-end…
  { pattern: /\b(full[\s-]?stack|fullstack|mean stack|mern|web application developer)\b/i, role: 'Full-Stack Developer…
  { pattern: /\b(ui[\s/.]?ux|user experience|product designer|ux designer|ui designer)\b/i, role: 'UI/UX Designer', la…
  { pattern: /\b(product manager|product owner|product management|roadmap)\b/i, role: 'Product Manager', label: 'produ…
  { pattern: /\b(data analyst|reporting analyst|bi analyst|business intelligence|dashboard)\b/i, role: 'Data Analyst',…
  { pattern: /\b(data scientist|predictive model|statistical model|machine learning|deep learning|ai engineer|mlops|nl…
  { pattern: /\b(devops|sre|site reliability|infrastructure engineer|platform engineer|ci\/cd|kubernetes|terraform)\b/…
  { pattern: /\b(cloud engineer|cloud architect|cloud infrastructure|aws|azure|google cloud)\b/i, role: 'Cloud Enginee…
  { pattern: /\b(android|ios|mobile developer|mobile app|flutter|react native)\b/i, role: 'Mobile App Developer', labe…
  { pattern: /\b(quality assurance|\bqa\b|test automation|automation engineer|sdet|software tester|test cases|test pla…
  { pattern: /\b(cyber ?security|information security|infosec|soc analyst|penetration|vulnerability|threat)\b/i, role:…
  { pattern: /\b(database admin|\bdba\b|data engineer|etl|data warehouse|database design|schema)\b/i, role: 'Database …
  { pattern: /\b(engineering manager|tech lead|team lead|head of engineering|\bcto\b|vp engineering|principal engineer…
  { pattern: /\b(technical writer|documentation specialist|docs writer|knowledge base)\b/i, role: 'Technical Writer', …
  { pattern: /\b(digital marketing|marketing manager|campaign|sem|ppc|adwords)\b/i, role: 'Digital Marketing Specialis…
  { pattern: /\b(content strategist|content marketing|content calendar|editorial)\b/i, role: 'Content Strategist', lab…
  { pattern: /\b(growth|performance marketing|conversion rate|a\/b test|funnel)\b/i, role: 'Growth Marketer', label: '…
  { pattern: /\b(sales|\bsdr\b|business development|\bbdr\b|account executive|pipeline|cold call|revenue)\b/i, role: '…
  { pattern: /\b(customer success|account manager|client success|renewal|onboarding)\b/i, role: 'Customer Success Mana…
  { pattern: /\b(recruiter|talent acquisition|sourcer|technical recruiter|hiring)\b/i, role: 'Recruiter / Talent Acqui…
  { pattern: /\b(financial analyst|finance|investment|equity research|financial model|budget)\b/i, role: 'Financial An…
  { pattern: /\b(project manager|program manager|delivery manager|scrum master|\bpmp\b)\b/i, role: 'Project Manager', …
  { pattern: /\b(hr\b|human resources|people ops|payroll|employee relations|generalist)\b/i, role: 'HR Generalist', la…
  { pattern: /\b(video editor|videographer|motion graphics|youtube|reels)\b/i, role: 'Video Editor', label: 'video edi…
  { pattern: /\b(graphic designer|visual designer|brand designer|illustrator|poster|logo)\b/i, role: 'Graphic Designer…
  { pattern: /\b(operations manager|logistics|supply chain|ops manager|inventory|procurement)\b/i, role: 'Operations M…
  { pattern: /\b(business analyst|requirements gathering|process improvement|systems analyst|stakeholder)\b/i, role: '…
  { pattern: /\b(technical support|it support|support engineer|customer support|helpdesk|troubleshoot)\b/i, role: 'Tec…
];

/** Roles that fit almost any background - surfaced when a resume has little signal. */
const WARM_TITLES = new Set([
  'Customer Success Manager',
  'HR Generalist',
  'Project Manager',
  'Business Analyst',
  'Technical Support Specialist',
  'Recruiter / Talent Acquisition',
  'Operations Manager',
  'Technical Writer',
]);

type Candidate = {
  role: RoleDefinition;
  matched: string[];
  hints: string[];
  percent: number;
};

/** "5 years of experience", "3+ yrs experience" ... */
function inferYears(text: string): number | null {
  const match = text.match(/(\d{1,2})\s*\+?\s*(?:years?|yrs?)\s*(?:of)?\s*(?:experience|exp)/i);
  if (match) return Number(match[1]);
  const alt = text.match(/(?:experience|exp)\D{0,20}(\d{1,2})\s*\+?\s*(?:years?|yrs?)/i);
  return alt ? Number(alt[1]) : null;
}

```

```
/**
 * Offline matcher used when no Grok API key is configured (or when the API
 * fails). Scores the role catalog from nextrole.json against the skills AND
 * job-title language found in the resume - and always returns a ranked list
 * with the closest-fitting role first.
 */
export function matchLocally(resumeText: string): AnalysisResult {
  const foundSkills = findSkills(resumeText);
  const foundSet = new Set(foundSkills.map((skill) => skill.toLowerCase()));

  // Roles implied by individual skills (skillRoles map in nextrole.json).
  const implied = new Map<string, string[]>();
  for (const skill of foundSkills) {
    for (const title of data.skillRoles[skill] ?? []) {
      const list = implied.get(title) ?? [];
      list.push(skill);
      implied.set(title, list);
    }
  }

  // Strong role signals from titles/phrases in the resume.
  const hints = new Map<string, string[]>();
  for (const hint of TITLE_HINTS) {
    const match = resumeText.match(hint.pattern);
    if (!match) continue;
    const list = hints.get(hint.role) ?? [];
    list.push(match[0].trim());
    hints.set(hint.role, list);
  }

  const candidates: Candidate[] = [];

  for (const role of data.roleCatalog) {
    const direct = role.skills.filter((skill) => foundSet.has(skill.toLowerCase()));
    const directSet = new Set(direct.map((skill) => skill.toLowerCase()));
    const viaSkills = (implied.get(role.title) ?? []).filter(
      (skill) => !directSet.has(skill.toLowerCase()),
    );
    const hintWords = hints.get(role.title) ?? [];
    if (direct.length === 0 && viaSkills.length === 0 && hintWords.length === 0) continue;
    const percent = Math.min(97, 56 + direct.length * 9 + viaSkills.length * 4 + hintWords.length * 7);
    candidates.push({ role, matched: [...direct, ...viaSkills], hints: hintWords, percent });
  }

  candidates.sort(
    (a, b) =>
      b.percent - a.percent ||
      b.hints.length - a.hints.length ||
      b.matched.length - a.matched.length,
  );

  const top = candidates.slice(0, 6);

  // Never leave the panel empty: pad with broader suggestions.
  if (top.length < 6) {
    const used = new Set(top.map((entry) => entry.role.title));
    const noSignal = hints.size === 0;
    const pool = noSignal
      ? [...data.roleCatalog].sort(
          (a, b) =>
            Number(WARM_TITLES.has(b.title)) - Number(WARM_TITLES.has(a.title)) ||
            b.skills.length - a.skills.length,
        )
      : [...data.roleCatalog].sort((a, b) => b.skills.length - a.skills.length);
    for (const role of pool) {
      if (used.has(role.title)) continue;
      top.push({
        role,
        matched: [],
        hints: [],
        percent: Math.max(42, 54 - top.length * 2),
      });
      used.add(role.title);
      if (top.length >= 6) break;
    }
  }

  // With no clear career signal, lead with roles that fit almost any background.
  if (hints.size === 0 && foundSkills.length <= 4) {
    top.sort(
      (a, b) =>
        Number(WARM_TITLES.has(b.role.title)) - Number(WARM_TITLES.has(a.role.title)) ||
        b.percent - a.percent,
    );
  }

  const roles: RoleMatch[] = top.map(({ role, matched, hints: hintWords, percent }) => {
    let why: string;
    if (matched.length > 0 && hintWords.length > 0) {
```

```
263|       why = `Your experience lines up through ${matched.slice(0, 4).join(', ')}, and your resume mentions
${hintWords[…
264|     } else if (matched.length > 0) {
265|       why = `Your experience lines up through ${matched.slice(0, 4).join(', ')}. ${role.summary}`;
266|     } else if (hintWords.length > 0) {
267|       why = `Your resume points at this role through ${hintWords.slice(0, 2).join(' and ')}.
${role.summary}`;
268|     } else {
269|       why = `A broader move worth exploring alongside your current experience. ${role.summary}`;
270|     }
271|     return {
272|       title: role.title,
273|       matchPercent: percent,
274|       seniority: role.seniority,
275|       summary: role.summary,
276|       why,
277|       skills: matched.length > 0 ? matched.slice(0, 6) : role.skills.slice(0, 4),
278|     };
279|   });
280|
281|   const years = inferYears(resumeText);
282|   const topHint = [...hints.entries()].sort((a, b) => b[1].length - a[1].length)[0];
283|
284|   let summary: string;
285|   if (foundSkills.length > 0) {
286|     summary = `We found ${foundSkills.length} skills in your resume, including ${foundSkills
287|       .slice(0, 5)
288|       .join(', ')}.`;
289|     if (years !== null) summary += ` About ${years} ${years === 1 ? 'year' : 'years'} of experience
noted.`;
290|     if (topHint) summary += ` Strongest signal: ${topHint[0]}.`;
291|   } else if (topHint) {
292|     summary = `Your resume reads strongest as ${topHint[0]}${
293|       years !== null ? `, with about ${years} ${years === 1 ? 'year' : 'years'} of experience` : ''
294|     }. We could not spot named skills, so matches below lean on that.`;
295|   } else {
296|     summary =
297|       'We read your experience but could not spot named skills, so the matches below are broader
suggestions.';
298|   }
299|
300|   return { skills: foundSkills.slice(0, 16), summary, roles };
301| }
302|
```

server/src/services/ocr.ts

```
  1| import path from 'node:path';
  2| import { createRequire } from 'node:module';
  3|
  4| const requireCjs = createRequire(__filename);
  5|
  6| /** Minimal structural types so we never depend on optional type packages. */
  7| type CanvasContext2D = {
  8|   fillStyle: string;
  9|   fillRect(x: number, y: number, width: number, height: number): void;
 10| };
 11| type CanvasSurface = {
 12|   width: number;
 13|   height: number;
 14|   getContext(kind: '2d'): CanvasContext2D;
 15|   toBuffer(mime: 'image/png'): Buffer;
 16| };
 17| type CanvasModule = { createCanvas(width: number, height: number): CanvasSurface };
 18|
 19| type PdfjsViewport = { width: number; height: number };
 20| type PdfjsRenderPage = {
 21|   getViewport(options: { scale: number }): PdfjsViewport;
 22|   render(options: Record<string, unknown>): { promise: Promise<void> };
 23|   cleanup(): void;
 24| };
 25| type PdfjsModule = {
 26|   getDocument(params: Record<string, unknown>): {
 27|     promise: Promise<{
 28|       numPages: number;
 29|       getPage(n: number): Promise<PdfjsRenderPage>;
 30|       destroy(): Promise<void>;
 31|     }>;
 32|   };
 33| };
 34|
 35| type TesseractWorker = {
 36|   recognize(image: Buffer): Promise<{ data: { text: string } }>;
 37|   terminate(): Promise<void>;
 38| };
 39|
```

```
/** Scanned resumes are slow - cap what we OCR per file. */
const MAX_OCR_PAGES = 8;
/** ~190 DPI; enough for resume-sized type without tripling OCR time. */
const RENDER_SCALE = 2.6;

/**
 * Language data ships in the @tesseract.js-data/eng package
 * (node_modules/@tesseract.js-data/eng/4.0.0/eng.traineddata.gz), so OCR
 * works fully offline - no CDN download at analysis time.
 */
function resolveLangPath(): string {
  const entry = requireCjs.resolve('@tesseract.js-data/eng');
  return path.join(path.dirname(entry), '4.0.0');
}

/**
 * OCR fallback for PDFs with no text layer (scanned or exported-as-image
 * resumes). Renders each page through pdf.js onto a node-canvas surface and
 * reads it with Tesseract's English model. Returns '' when the OCR stack is
 * unavailable or finds nothing, so callers can fall back to a clear error.
 */
export async function ocrPdf(bytes: Uint8Array): Promise<string> {
  const canvasModule = requireCjs('canvas') as CanvasModule;
  const pdfjs = requireCjs('pdfjs-dist/legacy/build/pdf.js') as PdfjsModule;
  const { createWorker } = requireCjs('tesseract.js') as {
    createWorker: (
      langs: string,
      oem: number,
      options: Record<string, unknown>,
    ) => Promise<TesseractWorker>;
  };

  const doc = await pdfjs.getDocument({
    data: new Uint8Array(bytes),
    isEvalSupported: false,
    disableFontFace: true,
    useWorkerFetch: false,
    verbosity: 0,
  }).promise;

  const pages: string[] = [];
  let worker: TesseractWorker | null = null;

  try {
    const limit = Math.min(doc.numPages, MAX_OCR_PAGES);
    for (let pageNumber = 1; pageNumber <= limit; pageNumber++) {
      const page = await doc.getPage(pageNumber);
      const viewport = page.getViewport({ scale: RENDER_SCALE });
      const surface = canvasModule.createCanvas(
        Math.ceil(viewport.width),
        Math.ceil(viewport.height),
      );
      const context = surface.getContext('2d');
      context.fillStyle = '#ffffff';
      context.fillRect(0, 0, surface.width, surface.height);
      await page.render({ canvasContext: context, viewport }).promise;
      const png = surface.toBuffer('image/png');
      page.cleanup();

      if (!worker) {
        worker = await createWorker('eng', 1, {
          langPath: resolveLangPath(),
          gzip: true,
          cacheMethod: 'none',
        });
      }
      const result = await worker.recognize(png);
      const text = (result.data.text ?? '').replace(/\n{3,}/g, '\n\n').trim();
      if (text) pages.push(text);
    }
  } finally {
    if (worker) await worker.terminate().catch(() => undefined);
    await doc.destroy().catch(() => undefined);
  }

  return pages.join('\n');
}
```

server/src/services/passwords.ts

```ts
import { randomBytes, scryptSync, timingSafeEqual } from 'node:crypto';

/**
 * Password hashing with Node's built-in scrypt (no extra dependency).
 * Stored format: "<salt-hex>:<scrypt-hash-hex>".
 */
const KEY_LENGTH = 64;

export function hashPassword(password: string): string {
  const salt = randomBytes(16).toString('hex');
  const hash = scryptSync(password, salt, KEY_LENGTH).toString('hex');
  return `${salt}:${hash}`;
}

/** Constant-time password check; false for malformed/missing hashes. */
export function verifyPassword(password: string, stored: string | undefined): boolean {
  if (!stored) return false;
  const [salt, hash] = stored.split(':');
  if (!salt || !hash) return false;
  const expected = Buffer.from(hash, 'hex');
  if (expected.length !== KEY_LENGTH) return false;
  const actual = scryptSync(password, salt, KEY_LENGTH);
  return timingSafeEqual(actual, expected);
}

```

server/src/store.ts

```ts
import fs from 'node:fs';
import { Types } from 'mongoose';
import { dataPath } from './data';
import type { AnalysisSource, RoleMatch } from './types';

/**
 * File-backed persistence inside the single `nextrole.json` data file.
 *
 * Used whenever MongoDB is not configured/connected, so sign-in, saved
 * analyses, and profile history keep working out of the box. When MongoDB is
 * available the Mongo models are used instead and this store stays untouched.
 *
 * Runtime data lives under the top-level `store` key:
 *   { "store": { "users": [...], "analyses": [...] } }
 */

export interface StoredUser {
  id: string;
  googleId?: string;
  name: string;
  email: string;
  picture?: string;
  passwordHash?: string;
  createdAt: string;
}

export interface StoredAnalysis {
  id: string;
  userId: string;
  fileName?: string;
  skills: string[];
  summary: string;
  roles: RoleMatch[];
  source: AnalysisSource;
  createdAt: string;
}

interface StoreShape {
  users?: StoredUser[];
  analyses?: StoredAnalysis[];
}

const MAX_ANALYSES = 100;

function readStore(): StoreShape {
  try {
    const parsed = JSON.parse(fs.readFileSync(dataPath, 'utf-8')) as { store?: StoreShape };
    return parsed.store ?? {};
  } catch (cause) {
    console.warn('[store] could not read nextrole.json store:', cause);
    return {};
  }
}

```

```
function writeStore(store: StoreShape): void {
  try {
    const file = JSON.parse(fs.readFileSync(dataPath, 'utf-8')) as Record<string, unknown>;
    file.store = store;
    const payload = `${JSON.stringify(file, null, 2)}\n`;

    // Windows (OneDrive, antivirus, the dev server) can hold short-lived read
    // locks on nextrole.json: rename needs delete access and fails with EPERM,
    // so retry with backoff and fall back to a direct overwrite.
    let lastError: unknown;
    for (let attempt = 0; attempt < 5; attempt++) {
      if (attempt > 0) {
        Atomics.wait(new Int32Array(new SharedArrayBuffer(4)), 0, 0, 80 * attempt);
      }

      const tmp = `${dataPath}.tmp`;
      try {
        fs.writeFileSync(tmp, payload, 'utf-8');
        fs.renameSync(tmp, dataPath); // atomic replace when possible
        return;
      } catch (cause) {
        lastError = cause;
        try {
          fs.unlinkSync(tmp);
        } catch {
          // Leftover tmp is harmless; next write reuses the name.
        }
      }

      try {
        fs.writeFileSync(dataPath, payload, 'utf-8'); // lock-tolerant fallback
        return;
      } catch (cause) {
        lastError = cause;
      }
    }
    throw lastError;
  } catch (cause) {
    console.warn('[store] could not write nextrole.json store:', cause);
  }
}

function newId(): string {
  return new Types.ObjectId().toString();
}

/** Finds (or creates) a user by Google id or email address. */
export function upsertUser(profile: {
  googleId?: string;
  name: string;
  email: string;
  picture?: string;
  passwordHash?: string;
}): StoredUser {
  const store = readStore();
  const users = store.users ?? [];
  const email = profile.email.trim().toLowerCase();
  const existing = users.find(
    (user) =>
      (profile.googleId && user.googleId === profile.googleId) ||
      user.email.toLowerCase() === email,
  );

  if (existing) {
    existing.name = profile.name;
    existing.email = profile.email;
    if (profile.picture !== undefined) existing.picture = profile.picture;
    if (profile.googleId) existing.googleId = profile.googleId;
    if (profile.passwordHash && !existing.passwordHash) existing.passwordHash = profile.passwordHash;
    writeStore(store);
    return existing;
  }

  const created: StoredUser = {
    id: newId(),
    ...(profile.googleId ? { googleId: profile.googleId } : {}),
    name: profile.name,
    email: profile.email,
    ...(profile.picture ? { picture: profile.picture } : {}),
    ...(profile.passwordHash ? { passwordHash: profile.passwordHash } : {}),
    createdAt: new Date().toISOString(),
  };
  users.push(created);
  store.users = users;
  writeStore(store);
  return created;
}

/** Finds a user by email address (used by password sign-in). */
```

```
export function findUserByEmail(email: string): StoredUser | undefined {
  const target = email.trim().toLowerCase();
  return (readStore().users ?? []).find((user) => user.email.toLowerCase() === target);
}

/** Stores (or replaces) a password hash for an existing user. */
export function setUserPasswordHash(userId: string, passwordHash: string): void {
  const store = readStore();
  const user = (store.users ?? []).find((entry) => entry.id === userId);
  if (!user) return;
  user.passwordHash = passwordHash;
  writeStore(store);
}

/** Saves an analysis for a user; newest first, capped at MAX_ANALYSES. */
export function saveAnalysis(
  userId: string,
  input: {
    fileName?: string;
    skills: string[];
    summary: string;
    roles: RoleMatch[];
    source: AnalysisSource;
  },
): StoredAnalysis {
  const store = readStore();
  const created: StoredAnalysis = {
    id: newId(),
    userId,
    ...(input.fileName ? { fileName: input.fileName } : {}),
    skills: input.skills,
    summary: input.summary,
    roles: input.roles,
    source: input.source,
    createdAt: new Date().toISOString(),
  };
  const analyses = store.analyses ?? [];
  analyses.unshift(created);
  store.analyses = analyses.slice(0, MAX_ANALYSES);
  writeStore(store);
  return created;
}

/** Lists a user's saved analyses, newest first. */
export function listAnalyses(userId: string): StoredAnalysis[] {
  const analyses = readStore().analyses ?? [];
  return analyses
    .filter((analysis) => analysis.userId === userId)
    .sort((a, b) => (a.createdAt < b.createdAt ? 1 : -1))
    .slice(0, 30);
}

/** Deletes one analysis owned by the user. Returns true when removed. */
export function deleteAnalysis(userId: string, id: string): boolean {
  const store = readStore();
  const analyses = store.analyses ?? [];
  const next = analyses.filter((analysis) => !(analysis.id === id && analysis.userId === userId));
  if (next.length === analyses.length) return false;
  store.analyses = next;
  writeStore(store);
  return true;
}

```

server/src/types.ts

```
export type RoleMatch = {
  title: string;
  matchPercent: number;
  seniority: string;
  summary: string;
  why: string;
  skills: string[];
};

export type AnalysisResult = {
  skills: string[];
  summary: string;
  roles: RoleMatch[];
};

export type AnalysisSource = 'grok' | 'local';

```

server/tsconfig.json

```
 1| {
 2|   "compilerOptions": {
 3|     "target": "ES2021",
 4|     "module": "CommonJS",
 5|     "moduleResolution": "node",
 6|     "lib": ["ES2021"],
 7|     "outDir": "dist",
 8|     "rootDir": "src",
 9|     "strict": true,
10|     "esModuleInterop": true,
11|     "skipLibCheck": true,
12|     "resolveJsonModule": true,
13|     "forceConsistentCasingInFileNames": true,
14|     "types": ["node"]
15|   },
16|   "include": ["src"]
17| }
18|
```